



T.C.

BARTIN ÜNİVERSİTESİ

MÜHENDİSLİK, MİMARLIK VE TASARIM FAKÜLTESİ

BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ

BİTİRME PROJESİ

**YAPAY ZEKA DESTEKLİ LOG VE DAVRANIŞ ANALİZİ İLE ANOMALİ
TESPİTİ**

21010310057

Hüseyin Arda HOŞCAN

DANIŞMAN

DOÇ. DR. Eyüp Burak CEYHAN

BARTIN-2025

ÖN SÖZ

Bu tez Bartın Üniversitesi Bilgisayar Mühendisliği Bölümü, bitirme projesi kapsamında gerçekleştirilen çalışmayı, akademik bir düzeyde açıklamak ve projenin sistematığını ortaya koymak amacıyla yazılmıştır. Bu projesinin hazırlanma sürecinde, proje fikrini benimle paylaşan, yol gösteren ve desteklerini esirgemeyen danışmanım Doç. Dr. Eyüp Burak Ceyhan’a en içten teşekkürlerimi sunarım. Kendisinin değerli katkıları ve yönlendirmeleri olmasaydı, bu projenin ortaya çıkması mümkün olamazdı.

21010310057

Hüseyin Arda HOŞCAN

ÖZET

YAPAY ZEKA DESTEKLİ LOG VE DAVRANIŞ ANALİZİ İLE ANOMALİ TESPİTİ

21010310057

Hüseyin Arda HOŞCAN

Bartın Üniversitesi

Mühendislik, Mimarlık ve Tasarım Fakültesi

Bilgisayar Mühendisliği Bölümü

Proje Danışmanı: **Doç. Dr. Eyüp Burak CEYHAN**

Bartın-2025, sayfa: 40

Günümüzün dijital dünyasında, teknolojinin sağladığı kolaylıklar beraberinde siber güvenlik tehditlerinde de karmaşık bir artışa yol açmıştır. Ticari kuruluşlar, devletler ve bireyler; kötü amaçlı yazılımlar, kimlik avı saldırıları, gizlilik ihlalleri, veri sızıntıları ve gelişmiş kalıcı tehditler gibi çeşitli siber saldırılara karşı her zamankinden daha savunmasızdır. Bu durum, geleneksel güvenlik yöntemlerinin yetersiz kaldığı ve makine öğrenmesi tabanlı çözümlere olan ihtiyacın kritik hale geldiği bir dönemi işaret etmektedir. Bu proje, yapay zeka tabanlı bir anomali tespit sistemi geliştirerek bu tehditlere karşı aktif bir koruma sağlamayı amaçlamaktadır.

Projenin temel amacı, Hadoop Dağıtılmış Dosya Sistemi (HDFS) loglarındaki anomalileri etkin bir şekilde tespit ederek, bu sistemlerin güvenliğini ve operasyonel sürekliliğini güvence altına almaktır. Siber saldırılar, geleneksel imza tabanlı yöntemlerle tespit edilmesi zor olan, sürekli evrilen ve gizlenen bir yapıya sahiptir. Bu saldırılar, sistemlere ve verilere ciddi zararlar verebilir, operasyonları aksatabilir ve bir kuruluşun itibarını zedeleyebilir. Bu nedenle, bu tehditlere karşı koymak ve güvenlik açıklarını kapatmak için daha gelişmiş ve akıllı çözümlere ihtiyaç duyulmaktadır. Bu proje, yapay zeka ve makine öğrenmesi tekniklerini kullanarak siber güvenlik tehditlerini tespit etmek için yenilikçi bir yaklaşım

sunmaktadır.

Projenin en yenilikçi yönü, çoklu makine öğrenmesi algoritmalarını (XGBoost, LightGBM, CatBoost, Random Forest ve KNN) entegre ederek hibrit bir yaklaşım sunmasıdır. Bu hibrit yaklaşım, her bir algoritmanın güçlü yanlarını birleştirerek, tek bir modelin potansiyel sınırlılıklarını aşmayı hedeflemiştir. Özellikle blok bazlı özellik çıkarımı metodu, ham HDFS log verisini işlenebilir bir formata dönüştürerek model performansını önemli ölçüde artırmıştır. Bu metot ile her bir log bloğundan toplamda 40 istatistiksel, zamansal ve sözcüksel özellik (örneğin, bir bloktaki benzersiz kelime sayısı, içerikteki belirli anahtar kelimelerin varlığı) çıkarılmıştır. Bu algoritmalar, yüksek doğruluk oranları ve karmaşık veri setlerini işleyebilme yetenekleriyle bilinir, bu da onları anomali tespiti için ideal adaylar haline getirmektedir.

Projede kullanılan veri seti, 11 milyondan fazla log kaydı içeren ve toplam 1.47 GB boyutuyla 38,7 saatlik gerçek bir HDFS çalışmasının sonucunda elde edilmiştir. Bu veri seti, HDFS'nin işleyişi sırasında oluşan olayları ve hataları kaydetmektedir. Her bir log kaydı, işlem ID'si, seviye, içerik ve blok ID'si gibi bilgiler içermektedir. Veri gizliliği ve etik konularına özen gösterilerek, log girdileri kişisel bilgileri içermeyecek şekilde anonimleştirilmiştir.

Araştırma bulgularına göre, eğitilen tüm modeller anomali tespiti konusunda yüksek doğruluk oranları ve F1-skorları sergilemiştir. Tekil modeller arasında XGBoost yüksek performans gösterse de farklı veri setleri üzerinde tutarlı sonuçlar vermeme potansiyeli taşımaktadır. Ancak, hibrit yaklaşımın, farklı veri setleri üzerinde dahi en güvenilir ve en tutarlı sonuçları verdiği kanıtlanmıştır. Hibrit model, en düşük yanlış negatif (FN) sayısı ile gerçek anomalilerin neredeyse tamamını tespit ederek, bir siber güvenlik sistemi için en kritik olan duyarlılık (recall) metriklerinde %99,91'in üzerinde bir başarı elde etmiştir. Bu durum, operasyonel sistemlerde yaşanabilecek ciddi sorunların önlenmesi açısından projenin ne kadar değerli olduğunu göstermektedir. Ayrıca, tüm modellerin eğitim ve test verileri arasındaki performans farkının minimum düzeyde olması, aşırı öğrenme (overfitting) problemi yaşanmadığını ve modellerin yeni veriler üzerinde de yüksek performans sergileyeceğini kanıtlamıştır.

Bu araştırmanın sonuçları, HDFS log verisi üzerinde anomali tespiti için geliştirilen hibrit modelin, günümüzün karmaşık siber güvenlik tehditlerine karşı etkin, güvenilir ve iyi genelleme yeteneğine sahip bir çözüm sunduğunu ortaya koymuştur. Projenin somut çıktıları arasında, sistemin yöneticilerin anomalileri ve sistem durumunu izlemesini sağlayan kullanıcı dostu bir grafik arayüzü ve düzenli olarak anomali istatistiklerini içeren raporlar

(PDF ve Excel formatında) bulunmaktadır. Gelecekte, daha gelişmiş doğal dil işleme (NLP) yöntemlerinin veya gerçek zamanlı veri akışı sistemlerinin entegre edilmesiyle projenin kapsamı ve performansı daha da artırılabilir.

Anahtar Kelimeler: Makine Öğrenmesi (Machine Learning), Anomali Tespiti, Siber Güvenlik, Veri Gizliliği, Hibrid Algoritmalar, Random Forest, KNN (K En Yakın Komşu), XGBoost, LightGBM, CatBoost, HDFS (Hadoop Distributed File System), Log Verisi Analizi, Zaman Bazlı Özellikler, Veri Ön İşleme, Tokenizasyon, TF-IDF, Gerçek Zamanlı Anomali Tespiti, Eğitim Verisi, F1 Skoru, Hiper parametre Ayarlama, Siber Tehditler, Raporlama, Tkinter Kullanıcı Arayüzü, Anomali Etiketleri, PDF Raporları.

ABSTRACT

ANOMALY DETECTION WITH AI-ASSISTED LOG AND BEHAVIOR ANALYSIS

21010310057

Hüseyin Arda HOŞCAN

Bartın University

Faculty of Engineering, Architecture and Design

Department of Computer Engineering

Project Advisor: Unvan Danışman Adı

Bartın-2025, pp: 40

The rapid advancement of technology has brought a corresponding increase in the complexity of cybersecurity threats. Today, commercial organizations, governments, and individuals are more vulnerable than ever to various cyberattacks, including malware, phishing, privacy breaches, data leaks, and advanced persistent threats. The growing sophistication of these threats, coupled with the inadequacy of traditional security methods, has made machine learning-based solutions critically important. This project aims to provide proactive protection against these threats by developing an AI-based anomaly detection system.

The core objective of this project is to effectively detect anomalies within Hadoop Distributed File System (HDFS) logs to ensure the security and operational continuity of these systems. Cyberattacks often have an evasive nature, making them difficult to detect with traditional signature-based methods. These attacks can cause significant damage to systems and data, disrupt operations, and lead to reputational loss. Therefore, more advanced and intelligent solutions are required to counter these threats and close security gaps. This project offers an innovative approach to detecting cybersecurity threats by leveraging

artificial intelligence and machine learning techniques.

The most innovative aspect of this project is its hybrid approach, which integrates multiple machine learning algorithms (XGBoost, LightGBM, CatBoost, Random Forest, and KNN). This hybrid model is designed to overcome the limitations of a single algorithm by combining the strengths of each. A block-based feature extraction method was developed to transform raw HDFS log data into a processable format, significantly improving model performance. This method extracts 40 statistical, temporal, and lexical features from each log block, such as the number of unique words and the presence of specific keywords within the content. The selected algorithms are known for their high accuracy and ability to process complex datasets, making them ideal candidates for anomaly detection.

The dataset used in the project consists of over 11 million log records, totaling 1.47 GB, collected from a real 38.7-hour HDFS operation. Each log record contains information such as a transaction ID, log level, content, and block ID. To ensure data privacy and adhere to ethical considerations, all log entries were anonymized to prevent the inclusion of personal information.

Our research findings show that all trained models demonstrated high accuracy and F1-scores in anomaly detection. While individual models like XGBoost performed well, they showed a potential for inconsistent results across different datasets. In contrast, the hybrid approach was proven to yield the most reliable and consistent results, even on varying datasets. The hybrid model achieved a recall of over 99.91%, successfully detecting almost all true anomalies with the lowest number of false negatives (FN). This highlights the project's value in preventing serious issues in operational systems. Furthermore, the minimal difference in performance between the training and test data confirmed that the models did not suffer from overfitting and can exhibit high performance on new, unseen data.

The results of this research demonstrate that the hybrid model developed for HDFS log anomaly detection provides an effective, reliable, and well-generalizing solution against today's complex cybersecurity threats. Tangible outputs of the project include a user-friendly graphical interface that allows system administrators to monitor anomalies and system status, along with automated reports in PDF and Excel formats containing anomaly statistics. Future work could involve integrating more advanced Natural Language Processing (NLP) methods or real-time data streaming systems to further enhance the project's scope and performance.

Keywords: Machine Learning, Anomaly Detection, Cyber Security, Data Privacy,

Hybrid Algorithms, Random Forest, KNN (K-Nearest Neighbors), XGBoost, LightGBM, CatBoost, HDFS (Hadoop Distributed File System), Log Data Analysis, Time-Based Features, Data Preprocessing, Tokenization, TF-IDF, Real-Time Anomaly Detection, Training Data, F1 Score, Hyperparameter Tuning, Cyber Threats, Reporting, Tkinter User Interface, Anomaly Label, PDF Reports.

İÇİNDEKİLER

İçindekiler

ÖN SÖZ.....	ii
ÖZET.....	iii
ABSTRACT	vi
İÇİNDEKİLER.....	ix
SİMGELER VE KISALTMALAR DİZİNİ.....	xi
1. GİRİŞ.....	1
2. LİTERATÜR ÖZETİ	3
3. MATERYAL VE METOT / YÖNTEM.....	5
3.1 Kullanılan Materyaller	5
3.1.1. Yazılım ve Programlama Dili	5
3.1.2. Veri İşleme ve Analiz Kütüphaneleri.....	6
3.1.3. Makine Öğrenmesi ve Gradient Boosting Framework'leri	6
3.1.4. Model Serialization ve Dağıtım	7
3.1.5. Görselleştirme Kütüphaneleri	8
3.1.6. Geliştirme Ortamı ve Diğer Araçlar.....	8
3.1.7. Donanım.....	8
3.1.8. Kullanıcı Arayüzü (GUI) Geliştirme Araçları	8
4. UYGULAMA VE GERÇEKLEŞTİRİM	13
4.1 Kod Yapısı	14
4.2 Proje geliştirme döngüsü.....	21
5. BULGULAR VE TARTIŞMA (TESTLER VE SONUÇLAR).....	23
5.1 Uygulamanın Test Senaryoları ve Doğrulama Adımları	23
5.2 Performans Ölçümleri, Hata Oranları ve Başarı Yüzdeleri	23
6. SONUÇ VE ÖNERİLER	30
7. BİRLEŞMİŞ MİLLETLER SÜRDÜRÜLEBİLİR KALKINMA AMAÇLARI İLE İLİŞKİLENDİRME.....	33
Birleşmiş Milletler Sürdürülebilir Kalkınma Amaçları Listesi ve Açıklamaları:.....	33
Çalışmanın Birleşmiş Milletler Sürdürülebilir Kalkınma Amaçları Kapsamında Değerlendirilmesi.....	35
7.1 Amaç 8: İnsana Yakışır İş ve Ekonomik Büyüme (Decent Work and Economic Growth):.....	36

7.2 Amaç 9: Sanayi, Yenilikçilik ve Altyapı (Industry, Innovation and Infrastructure):

36

KAYNAKLAR..... 37

SİMGELER VE KISALTMALAR DİZİNİ

KISALTMALAR

HDFS	: Hadoop Distributed File System- Hadoop Dağıtılmış Dosya Sistemi
ML	: Machine learning/ Makine öğrenmesi
GUI	: Grafiksel Kullanıcı Arayüzü (Graphical User Interface)
KNN	: K En Yakın Komşu (K-Nearest Neighbors)
XGBoost	: Ekstrem Gradyan Artırma (Extreme Gradient Boosting)
LightGBM	: Hafif Gradyan Artırma Makinesi (Light Gradient Boosting Machine)
CatBoost	: Kategorik Artırma (Categorical Boosting)
TF-IDF	: Terim Sıklığı-Ters Belge Sıklığı
FN	: Yanlış Negatif (False Negative)
FP	: Yanlış Pozitif (False Positive)
PDF	: Taşınabilir Belge Formatı (Portable Document Format)
BM SKA	: Birleşmiş Milletler Sürdürülebilir Kalkınma Amaçları
TP	: Gerçek Pozitif (True Positive)
TN	: Gerçek Negatif (True Negative)
Vd.	:Ve diğerleri
v.	:Versiyon

1. GİRİŞ

Günümüzde teknolojinin hızla ilerlemesi, beraberinde siber güvenlik tehditlerinde de bir artışa neden olmuştur. Ticari kuruluşlar, devletler ve bireyler; kötü amaçlı yazılımlar, kimlik avı saldırıları, gizlilik ihlalleri, veri sızıntıları ve gelişmiş kalıcı tehditler gibi çeşitli siber saldırılara karşı savunmasız hale gelmiştir yani günümüzde siber güvenlik tehditlerinin artan karmaşıklığı ve geleneksel yöntemlerin yetersiz kalması, makine öğrenmesi tabanlı çözümlere olan ihtiyacı kritik hale getirmiştir.

Bu proje, yapay zeka tabanlı bir anomali tespit sistemi geliştirerek bu tehditlere karşı aktif bir koruma sağlamayı amaçlamaktadır. Siber saldırılar, geleneksel güvenlik yöntemleriyle tespit edilmesi zor olan karmaşık ve sürekli gelişen bir yapıya sahiptir. Ayrıca teknolojinin gelişmesiyle geleneksel olarak yapılan bireyin incelemesine dayanan şu anki yapı yüksek süre zaman kayıplarına yol açmaktadır. Günümüzde zamanın kıymeti paha biçilemezdir. Bu saldırılar, sistemlere ve verilere ciddi zararlar verebilir, operasyonları aksatabilir ve itibar kaybına yol açabilir. Dolayısıyla, bu tehditlere karşı koymak ve güvenlik açıklarını kapatmak için daha gelişmiş ve akıllı çözümlere ihtiyaç duyulmaktadır. Bu proje, yapay zeka ve makine öğrenmesi tekniklerini kullanarak siber güvenlik tehditlerini tespit etmek için yenilikçi bir yaklaşım sunmaktadır. Projenin yenilikçi yönü, çoklu makine öğrenmesi algoritmalarını (XGBoost, LightGBM, CatBoost, Random Forest ve KNN) entegre ederek hibrit bir yaklaşım sunması ve etiketsiz veriler üzerinde bile %99,79 gibi yüksek bir oranlı bir doğrulukla anomali tespiti yapabilmesidir. Bu algoritmalar, yüksek doğruluk oranları ve karmaşık veri setlerini işleyebilme yetenekleriyle bilinir. Ayrıca projede geliştirilen blok bazlı özellik çıkarımı metodu, HDFS loglarındaki Ham veriyi işlenebilir 40 istatistiksel, zamansal özelliğe dönüştürerek model performansını arttırmaktadır. Bu özellikler “Log mesajındaki kelime sayısı, Log mesajındaki benzersiz kelime sayısı, Log mesajının toplam karakter sayısı, Log mesajındaki sayısal değerlerin sayısı, Log mesajındaki özel karakterlerin sayısı, Log mesajının içerdiği "error", "fail", "warning" gibi anahtar kelimelerin varlığı (binary), Log mesajındaki noktalama işaretlerinin sayısı, Log seviyesi (INFO, DEBUG, ERROR vb.) için tekil ID, Log şablonunun (event template) ID'si, Log şablonunun toplam frekansı, Log şablonunun görelilik frekansı (toplam olaya oranı), Log mesajındaki büyük harfli kelime sayısı, Bir önceki log olayından bu yana geçen süre (milisaniye cinsinden), Log olayının günün hangi saatinde meydana geldiği, Log olayının haftanın hangi gününe denk geldiği, Log olayının günün kaçınıcı dakikasında meydana geldiği, Log olayları arasındaki ortalama zaman farkı (bir pencere içinde), Log olayları arasındaki zaman farklarının standart sapması, Bir oturumun veya bloğun toplam süresi,

Oturum süresine göre log olaylarının frekansı (saniyedeki log sayısı), Bir HDFS bloğu veya oturumu içindeki toplam olay sayısı, Oturum içindeki benzersiz olay şablonu sayısı, Oturumdaki olay sayısına göre benzersiz olay sayısının oranı, Oturum içindeki en sık görülen olayın frekansı, En sık görülen olayın oturumdaki toplam olaya oranı, Her bir olayın oturum içindeki pozisyonu (ilk, ikinci, sonuncu vb.), Belirli bir olay şablonu (örneğin "data_replicate") için toplam sayım, Belirli bir olayın oturum içindeki ilk kez görülme süresi, Belirli bir olayın oturum içindeki son kez görülme süresi, Oturumda yer alan olay şablonlarının toplam sayısı (vektör), Olay şablonları arasındaki geçişlerin frekansı (bigram), Olay şablonları arasındaki geçişlerin frekansı (trigram), Oturumun başından itibaren geçen zamana göre log olaylarının frekansı, Oturumdaki her bir olayın TF-IDF skoru (terim frekansı-ters belge frekansı), Oturumdaki olayların sırası, Oturumdaki olayların her birinin bir önceki olaya göre konumu, Oturumdaki olayların, bir önceki olaydan sonra kaçınıcı sırada olduğu, Oturumdaki olayların sırasının entropisi, Oturumdaki olayların şablonlarının bir dize olarak birleştirilmesi, Bir blok veya oturumda "error" kelimesini içeren log sayısının toplam log sayısına oranı” şeklindedir. Sistemin bir diğer kritik bileşeni, ham log verilerinin işlenmesi ve anlamlı özelliklere dönüştürülmesi aşamasında kullanılan esnek regex yapısıdır. Geleneksel, sabit bir yapı aksine, bu dinamik regex yapısı farklı formattaki ve kaynaklardaki log satırlarını (örneğin, farklı HDFS dağıtımlarından veya sistemlerden gelen loglar) otomatik olarak tanıyıp ayrıştırabilmektedir. Bu esneklik, sistemin hızla değişen siber tehdit ortamına ve çeşitli log yapılarına kolayca uyum sağlamasına olanak tanır, böylece modelin genelleme yeteneğini ve gerçek dünya uygulanabilirliğini artırır. Projede kullanılacak veri seti [23], 11 milyondan fazla log kaydı içeren; 1,47 Gb boyutu olan ve 38,7 saatlik çalışmanın sonucu elde edilmiş HDFS log veri setidir. Bu veri seti, HDFS'nin işleyişi sırasında oluşan olayları ve hataları barındırır. Her log kaydı; tarih ve saat, işlem ID'si, seviye, bileşen, içerik, olay ID'si ve şablonu, parametre listesi, blok ID'si ve anomali etiketi gibi bilgiler içerir. Veri gizliliği ve etik konularına özen gösterilerek, log girdileri kişisel bilgileri içermeyecek şekilde anonimleştirilmiştir.

Bu raporun ikinci bölümünde konuyla ilgili literatür özeti sunulmuştur. Üçüncü bölümde sistem mimarisi ve kullanılan yöntemler açıklanmakta, dördüncü bölümde test sonuçları ve performans analizleri yer almaktadır. Beşinci ve son bölümde ise genel değerlendirme ve gelecekte yapılabilecek çalışmalar tartışılmıştır.

2. LİTERATÜR ÖZETİ

Siber tehditlerin artan yapısı nedeniyle, geleneksel imza tabanlı güvenlik sistemleri yetersiz kalmaya başlamıştır. Bu nedenle araştırmacılar makine öğrenmesi (ML) tabanlı anomali tespiti çözümlerine yönlendirmiştir. Bu bölümde, siber güvenlikte anomali tespiti, log analizi ve bu projede kullanılan makine öğrenmesi yöntemleri ile ilgili mevcut literatür özetlenmektedir.

Geleneksel güvenlik duvarları ve saldırı tespit sistemleri (IDS), önceden tanımlanmış imzalara dayanır ve sıfıncı gün (zero-day) saldırılarını tespit etmekte başarısız olur. Geçmişten günümüze bakacak olursak Chandola [25] vd., anomali tespitinin temel kavramlarını ve çeşitli uygulama alanlarını kapsamlı bir şekilde incelemiş, gözetimli ve gözetimsiz öğrenme yöntemlerinin bu alandaki önemine ilk vurgu yapan kişi olarak tarihe geçmiştir. Bu proje de aynı şekilde anomali tespit sisteminin öneminin altını çizmektedir.

Sistem logları, ağ ve yazılım davranışı hakkında bilgiler sunan en derin kaynaktır. Ancak bu log verilerinin boyutu doğrudan analiz sürecini zorlaştırmaktadır. Xie, M.[24] vd. gibi araştırmacılar, log anahtarları ve parametrelerini ayırmak için düzenli ifadeler (regex) kullanmanın önemini vurgulamışlardır. Bu, literatürde sıklıkla log ayrıştırma (log parsing) adımlarıyla gerçekleştirilmiş ve ardından oturum (session) veya blok bazlı özellik çıkarımı yapılmıştır. Bu projede benimsenen blok bazlı özellik çıkarımı yaklaşımı, M. Goldstein ve A. Dengel[21] (2012)'nin Histogram-Based Outlier Score [(HBOS) [25] çalışması gibi, verinin global istatistiksel dağılımını temel alan yöntemlerle benzerlik göstermekte olup, log dizilerindeki anormal davranışları istatistiksel özelliklerle yakalamayı amaçlamaktadır. Literatürde, siber güvenlik için çok çeşitli ML algoritmaları incelenmiştir. Sommer, R. ve Paxson, V. [13], ağ trafiği analizinde geleneksel ML yöntemlerinin zorluklarını tartışmışlardır. Daha yakın tarihli çalışmalar, gelişmiş ensemble yöntemlerine odaklanmıştır. Random Forest algoritması, L. Breiman [2] tarafından önerilmiş olup, birçok çalışmada (örneğin, A. Shiravi [12] vd.) sağlamlığı ve yüksek doğruluğu nedeniyle ağ saldırılarını tespit etmek için kullanılmıştır. XGBoost [T. Chen [4] ve C. Guestrin], LightGBM [G. Ke vd. [8] ve CatBoost [L. Prokhorenkova vd.[11]] gibi gradient boosting algoritmaları, kazanımlar hızı ve tahmin performansı açısından birçok Kaggle yarışmasında ve akademik çalışmada en iyi sonuçları vermiştir. Özellikle dengesiz veri kümelerinde (normal oranı, anormal olandan çok daha fazladır) sınıflandırma için güçlü bir yöntemdir. K-En Yakın Komşu (KNN) gibi daha basit algoritmalar ise [S. Garcia vd. [6]] gibi çalışmalarda, seyrek veri noktalarındaki doğru tespit özelliği için bu projede kullanılmıştır.

Tekil modellerin sınırlı olan yapısından ötürü geçmişten bu zamana araştırmacılar hibrit

yaklaşımları denemişlerdir. [Ahammad, S. H. vd.[3]], birden fazla taban öğreniciyi bir araya getiren ensemble yöntemlerinin, tek bir modelin genelleme hatasını ve varyansını azaltarak genellikle daha yüksek ve daha kararlı doğruluk sağladığını göstermiştir. Bu proje düşük boyutlu loglarda kötü performans sağlayan Random forest ve büyük boyutlu loglarda kötü performans sağlayan KNN modellerinin eksik yönlerini kapatacağı düşünülerek hibrit bir yaklaşımda bulunmuştur.

Aynı veri seti kullanılarak yapılan çalışmalar incelendiğinde, HDFS1 veri seti üzerinde çeşitli yaklaşımlarla farklı doğruluk oranları elde edildiği görülmektedir. Örneğin, Loglizer [14] aracı kullanılarak uygulanan Invariants Mining yöntemi %87,2, PCA tabanlı yöntem %95,1 ve Random Forest algoritması %96,8 doğruluk oranlarına ulaşmıştır. Derin öğrenme tabanlı bir yaklaşım olan DeepLog[28], HDFS1 veri seti üzerinde %96,9 F1-skor elde ederken, bir diğer ileri seviye model LogBERT [9] %98,4 gibi oldukça yüksek bir performans sergilemiştir. WraySmith/log-anomaly [16] projesi, CNN tabanlı modeliyle %99'un üzerinde F-skor rapor etmiştir. LogLead çalışmasının karşılaştırmalı analizinde ise Isolation Forest %97,1, LSTM %98,3 ve Logistic Regression %95,6 doğruluk değerleriyle sonuçlanmıştır.

Bu proje, şu zamana kadar anlatılan çalışmalardan farklı olarak tek bir model yerine birçok farklı model ve hibrit yaklaşım sunmakta ve özellikle ön işleme aşamasında geliştirilen blok bazlı istatistiksel özellik çıkarımı yöntemi ile öne çıkmaktadır. Geliştirilen sistem, XGBoost algoritması ile %99,79 gibi son derece yüksek bir doğruluk oranına ulaşmıştır. LogBERT [9] (%98,4) ve DeepLog [28](%96,9) gibi karmaşık derin öğrenme modellerinin bile üzerinde olup, mevcut literatürdeki en yüksek doğruluk oranlarından birine karşılık gelmektedir. Ayrıca, bu çalışmada kullanılan CatBoost (%99,72), LightGBM (%99,68) ve Random Forest (%99,41) modelleri de tek başlarına dahi literatürdeki birçok çalışmadan daha yüksek başarı göstermiştir. Bu sonuçlar, geliştirilen özellik çıkarımı metodolojisinin etkinliğini ve seçilen ensemble algoritmalarının bu problemdeki üstün yeteneğini kanıtlamaktadır. Sonuç olarak, bu proje aynı veri seti üzerinde çalışan diğer araştırmalarla kıyaslandığında, hem kullandığı metodoloji (çoklu model karşılaştırması ve istatistiksel özellik çıkarımı) hem de elde ettiği %99,79'luk doğruluk oranı ile literatürdeki çalışmaları bir adım öteye taşımaktadır.

3. MATERYAL VE METOT / YÖNTEM

Teknolojinin ve siber güvenliğin hayatımızdaki yeri tartışılmaz bir gerçektir. Günümüz koşullarında önlem ve tespit sistemleri kullanılmaksızın teknolojik dünyada yolculuk yapmak büyük problemlere yol açmaktadır. Bu bölümde, yapay zeka tabanlı anomali tespit sistemi geliştirmek için kullanılan veri seti, veri ön işleme ve özellik mühendisliği adımları, denenen makine öğrenmesi algoritmaları ve deneysel düzenek ayrıntılı olarak açıklanmaktadır.

Çalışmada, bu alanda bir benchmark (kıyaslama ölçütü) olarak kabul edilen HDFS (Hadoop Distributed File System) Log Data veri seti kullanılmıştır. Bu veri setinde [23] 11,175,629 satır bulunmaktadır. Veri seti toplam boyutu 1.47 GB'dir. 38,7 saatlik bir çalışmanın ardından veri seti elde edilmiştir. Bu veri setinde toplam 40 adet öznitelik bulunmaktadır. Bu da toplam 575061 veri demek oluyor. Bu verilerin 16838 tanesi anomali olarak etiketlenmiştir. Toplam “558223” tanesi de normal olarak etiketlenmiştir. Yani yüzdelik olarak bakacak olursak yaklaşık %2,93'ü anomali olarak etiketli satırdan, %97.07'si normal olarak etiketli satırdan oluşmaktadır. Veri setinin %2,93 gibi düşük bir değer olması gerçek dünya senaryolarını yansıtmaktadır. Sağlıklı bir sistemde çoğunlukla normal işlemler gerçekleşir ve anormal durumlar nadiren oluşur. Ancak bu tür dengesiz dağılım, makine öğrenmesi modeli eğitimi için önemli bir zorluk oluşturmaktadır. Model, tüm veri tahminlerinde “normal” olarak tahmin etse bile %97'lik bir doğruluk oranı elde eder; ancak bu durum tespit edilmesi en mühim olan anomalilerin tamamen gözden kaçması anlamına gelir. Bu nedenle, bu çalışmada model değerlendirmesi için doğruluk (accuracy) metriğinin yanı sıra, özellikle azınlık sınıfının (anomaliler) performansını ölçen F1-Skor, Kesinlik (Precision) ve Duyarlılık (Recall) gibi metriklerin kullanılmıştır.

3.1 Kullanılan Materyaller

Projede materyal olarak yazılım araçları kullanılmıştır. Bunların en önemli olanları aşağıda başlıklar halinde açıklanmıştır. (“v.” Versiyonu bildirir ve “+” işareti o versiyon ve üstünde çalışılabileceğini belirtir).

3.1.1. Yazılım ve Programlama Dili

- Python (v3.9+): Projenin temel programlama dili olarak seçilmiştir. Basit sözdizimi, geniş kütüphane ekosistemi ve veri bilimi projelerine olan uygunluğu nedeniyle tercih edilmiştir.

3.1.2. Veri İşleme ve Analiz Kütüphaneleri

- Pandas (v1.3+): Ham log verisinin yüklenmesi, temizlenmesi, filtrelenmesi, bloklara gruplanması ve istatistiksel özelliklerin çıkarılması işlemlerinin tamamında ana veri yapısı (DataFrame) olarak kullanılmıştır.
- NumPy (v1.21+): Büyük ölçekli sayısal hesaplamaların, dizi işlemlerinin ve lineer cebir operasyonlarının verimli bir şekilde yapılmasını sağlamıştır.
- Re (Regex) Modülü: Ham log satırlarının ayrıştırılması (parsing) için büyük öneme sahiptir. Log mesajlarındaki blok ID'leri (blk_[\d]+), işlem türleri ve diğer parametreleri ayıklamak için esnek düzenli ifade (regex) desenleri kullanılmıştır.
- NLTK (Natural Language Toolkit- v3.7+): Log mesajlarının doğal dil işleme (NLP) teknikleriyle analiz edilmesi amacıyla kullanılmıştır. [Örnek: Kelime tokenizasyon, kelime köklerini bulma (stemming) ve stop word temizleme adımları kullanılmıştır].

3.1.3. Makine Öğrenmesi ve Gradient Boosting Framework'leri

Projede, performans karşılaştırması yapmak ve en iyi sonucu verecek modeli belirlemek amacıyla çeşitli makine öğrenmesi algoritmaları denenmiştir. Tüm algoritmaların uygulanması ve eğitimi için Scikit-learn (v1.0+) kütüphanesi temel alınmıştır. Kullanılan algoritmalar aşağıda listelenmiştir:

- XGBoost (v1.5+): Yüksek performanslı ve optimize edilmiş bir Gradient Boosting kütüphanesidir. Model doğruluğunu ve hızını artırmak amacıyla kullanılmıştır.
- LightGBM (v3.3+): Büyük veri kümeleri üzerinde hızlı eğitim süreleri ve düşük bellek kullanımı sağlayan bir Gradient Boosting framework'üdür.

- CatBoost (v1.0+): Kategorik özellikleri işlemedeki yeteneği ve varsayılan hiper parametrelerle bile yüksek performans göstermesi nedeniyle projeye dahil edilmiştir.
- Random Forest (Scikit-learn): klasik makine öğrenmesi yöntemlerinin en bilinen örneklerinden biridir. Çok sayıda karar ağacının oluşturduğu bir orman yapısıyla çalışır ve yüksek doğruluk ile model karmaşıklığına karşı aşırı öğrenmeye (overfitting) dirençli olması nedeniyle seçilmiştir. Ayrıca düşük boyutlu loglarda uzun süren tahmin safhası nedeniyle pek tercih edilmemektedir. Bu eksik yanını gidermek için KNN modeli ile hibrit bir yaklaşım ile kullanılmıştır.
- K-En Yakın Komşu (KNN) (Scikit-learn): Basit ve etkili bir sınıflandırma algoritmasıdır. Anomali tespiti problemi, "nadir sınıf" tespiti problemi olduğu için, KNN'in seyrek veri noktalarını tespit etme yeteneği onu değerli bir model yapmıştır. Model performansı karşılaştırmalarında referans olarak kullanılmıştır. Ayrıca modelin büyük boyutlu loglarda iyi performans göstermediği bilinmektedir. Bu nedenle Random forest ile kullanılarak hibrit bir yapı da daha iyi performans göstermiştir.
- Scikit-learn (v1.0+): Verilerin eğitim ve test kümelerine ayrılması (train_test_split), özellik ölçeklendirme (StandardScaler), makine öğrenmesi modelleri (XGBoost, Random Forest, KNN dahil) ve model performans metriklerinin (precision, recall, F1-Score) hesaplanması için kullanılmıştır.

3.1.4. Model Serialization ve Dağıtım

- Joblib (v1.1+): Eğitilmiş makine öğrenmesi modellerinin (özellikle büyük boyutlu NumPy dizileri içerenlerin) diske kaydedilmesi (Serialization) ve daha sonra tekrar yüklenmesi (Deserialization) için kullanılmıştır. Bu, modeli yeniden eğitmeye gerek kalmadan gelecekteki tahminler için kullanılabilmesini sağlamıştır.

3.1.5. Görselleştirme Kütüphaneleri

- Matplotlib (v3.5+): Veri dağılımlarının, özelliklerin histogramlarının ve model eğitim kayıplarının çizilmesinde kullanılmıştır.
- Seaborn (v0.11+): Daha estetik ve bilgilendirici istatistiksel grafikler (örneğin, korelasyon matrisi ısı haritası) oluşturmak için kullanılmıştır.

3.1.6. Geliştirme Ortamı ve Diğer Araçlar

- Jupyter Notebook/ Pycharm: Etkileşimli kod geliştirme, hızlı prototipleme ve sonuçların anında görselleştirilmesi için kullanılmıştır.
- Git & GitHub: Kod paylaşımı ve proje yönetimi için kullanılmıştır.

3.1.7. Donanım

İşlemci (CPU): AMD Ryzen 5 5600H

Bellek (RAM): 16 Gb DDR4 Ram

GPU: NVIDIA Geforce RTX 3050 Laptop GPU

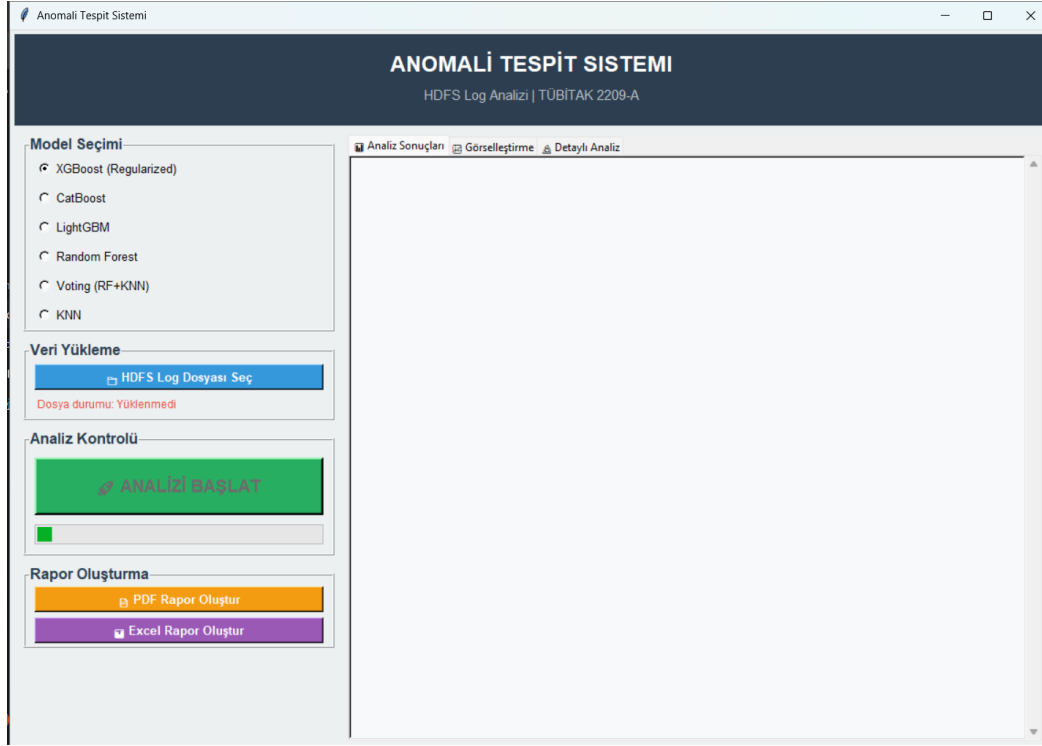
3.1.8. Kullanıcı Arayüzü (GUI) Geliştirme Araçları

Geliştirilen bu sistemin kullanılabilirliğini arttırmak ve kullanıcılar için erişilebilir hale getirmek için (GUI) grafiksel kullanıcı arayüzü tasarlanmıştır. Bu arayüz hem kullanıcıların Log dosyalarını yükleyebilmesini hem de analiz işlemini başlatıp sonuçlarını görebilmesini sağlamaktadır. Ayrıca bu sonuçlar görselleştirilmiş bir şekilde arayüzde bulunmaktadır. Bu şekilde kullanıcı rahatlıkla anomalileri ve nedenlerini inceleyebilir. Ayrıca kullanıcı

sonuçları Excel ve Word formatında raporlayabilir.

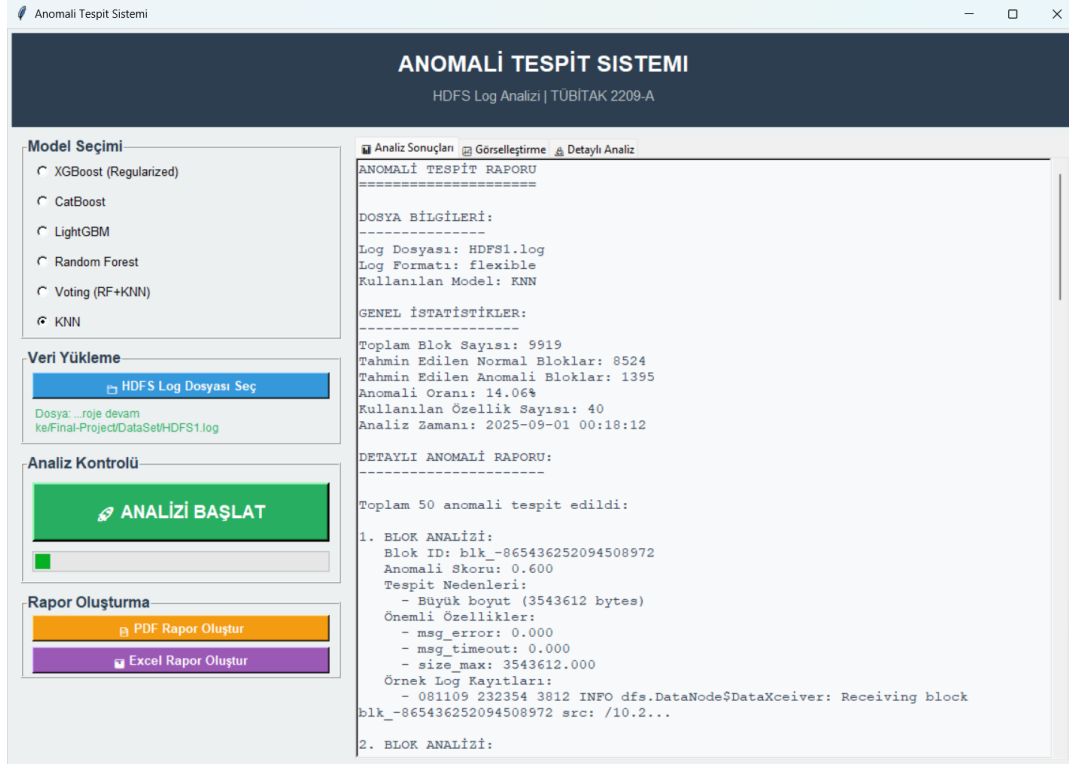
3.1.8.1 Uygulama Giriş Bölümü

Kullanıcı arayüzü sol ve sağ olmak üzere iki kısımdan oluşur. Sol kısımda model seçimi, veri yükleme kısmı, analiz kontrolü ve rapor oluşturma kısmı bulunur. Arayüzde istenilen HDFS dosyası (txt, csv, log formatında olabilir) yüklendikten ve model seçimi yapıldıktan sonra analizi başlat tuşuna basılarak analiz işlemi başlatılır.



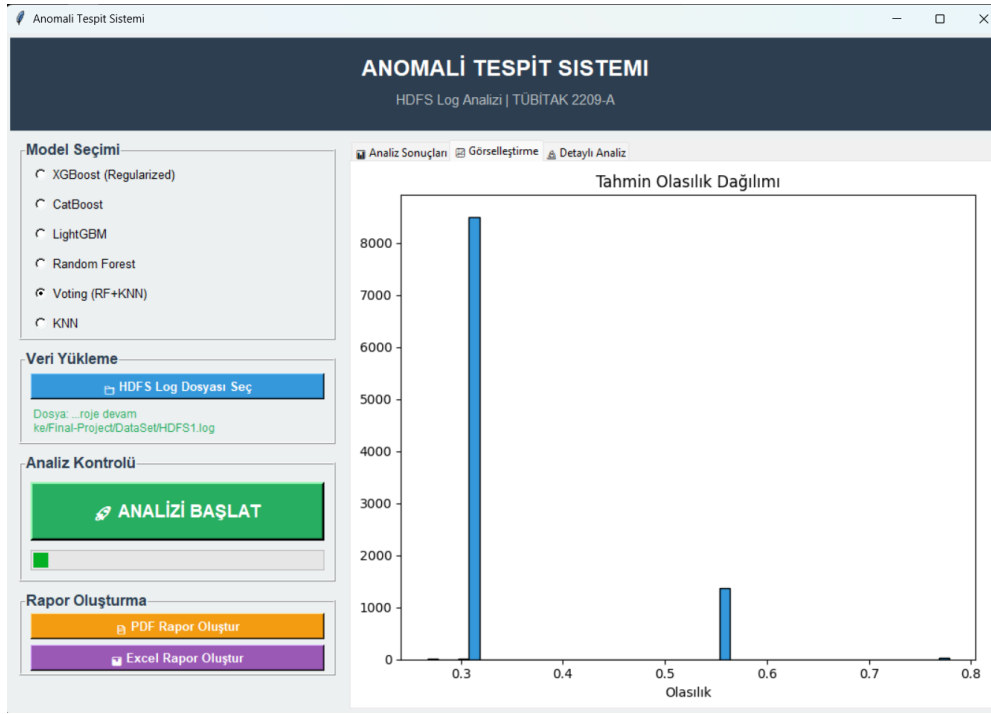
Görsel 1 Arayüz görüntüsü.

Görsel 1’de görüldüğü üzere kullanıcı arayüzü basit bir tasarıma sahiptir. Sağ bölümünde sonuçların gösterileceği ekran bulunur. “Analiz Sonuçları”, “Görselleştirme” ve “Detaylı Analiz” kısımları yer almaktadır.



Görsel 2 Arayüz Görüntüsü

Ham veri seçimi ve Model seçimi yapılmasının ardından “ANALİZİ BAŞLAT” yaşılan butona tıklanarak analiz süreci başlatılır. Analiz aşamasında, “Analiz kontrol birimi” altında yeşil bir imleç analiz aşaması boyunca gitme ve gelme hareketi yapar. Ardından görsel 2 de görüldüğü üzere analiz sonuçları ekrana gelir. Dosya bilgisi altında “Log Dosyası ismi”, “Log formatı (Flexible, standart)”, “Kullanılan model” kısımları yer alır. “GENEL İSTATİSTİKLER” kısmı altında “Toplam Blok Sayısı”, “Tahmin Edilen Normal Bloklar”, “Tahmin Edilen Anomali Bloklar”, “Anomali Oranı”, “Kullanılan Özellik Sayısı”, “Analiz Zamanı” kısımları yer almaktadır. “DETAYLI ANOMALİ RAPORU” kısmında “Toplam anomali tespit sayısı ve her bir bloğun blok analizi” bulunur.



Görsel 3 Arayüz Görüntüsü

Görsel 3'te tahmin olasılık dağılımını görmekteyiz. Bu veriler Pdf ve Excel formatında raporlanabilmektedirler.

ANOMALİ TESPİT SİSTEMİ
HDFS Log Analizi | TÜBİTAK 2209-A

Model Seçimi

- ☐ XGBoost (Regularized)
- ☐ CatBoost
- ☐ LightGBM
- ☐ Random Forest
- ☒ Voting (RF+KNN)
- ☐ KNN

Veri Yükleme

HDFS Log Dosyası Seç

Dosya: ...roje devam
ke\Final-Proje\DataSet\HDFS1.log

Analiz Kontrolü

ANALİZİ BAŞLAT

Rapor Oluşturma

PDF Rapor Oluştur

Excel Rapor Oluştur

Analiz Sonuçları | Görselleştirme | Detaylı Analiz

Metrik	Değer
Toplam Blok Sayısı	9919
Normal Bloklar	8524
Anomali Bloklar	1395
Anomali Oranı (%)	14.06
Özellik Sayısı	40
Tespit Edilen Anomaliler	50
Ortalama Anomali Skoru	0.616

Görsel 4 Arayüz Görüntüsü

Görsel 4’te bu metriklerin sayısal değerlerini görmektesiniz. “Detaylı Analiz” kısmı metriklerin sayısal olarak değerlerini gösterir.

3.1.8.2 Uygulama Raporlama Bölümü

Analiz aşaması ardından kullanıcı isterse analizi raporlayabilir.

DETAYLI ANOMALI TESPİT RAPORU

Rapor Bilgileri	
Oluşturulma Tarihi	2025-09-06 22:10:31
Log Dosyası	testData2.log
Kullanılan Model	KNN

Metrik	Değer
Toplam Blok Sayısı	6304
Normal Blok Sayısı	6303
Anomali Blok Sayısı	1
Anomali Oranı (%)	0.02
Kullanılan Özellik Sayısı	40

ANOMALI DETAYLARI

1. Blok Analizi

Blok ID	blk_1074130766
Anomali Skoru	1.000
Tespit Nedenleri	Yuksek Timestamp_avg_interval değeri (39561522.00), Yüksek LineNumber_sum değeri

Görsel 5 de Detaylı Anomali PDF Tespit Raporu Yer Almaktadır.

	A	B	C
1	Metrik	Değer	
2	Toplam Blok Sayısı	6304	
3	Normal Blok Sayısı	6303	
4	Anomali Blok Sayısı	1	
5	Anomali Oranı (%)	0.02	
6	Kullanılan Özellik Sayısı	40	
7	Log Dosyası	testData2.log	
8	Analiz Zamanı	2025-09-06 22:10:31	
9	Kullanılan Model	KNN	
10			

Görsel 6 de Detaylı Anomali Excel Tespit Raporu Yer Almaktadır.

Görsel 4'ün sol altında bulunan rapor oluşturma kısmına tıklandığında anomali raporu pdf şeklinde oluşur kullanıcı dilerse istediği bir konuma bu pdf'i kaydedebilir. Görsel 5'te görüldüğü üzere analiz sonuçları kısmında aktarılan tüm bilgiler bu pdf raporunda gözlemlenebilir. Görsel 6'da Excel formatında raporlanmış kısım yer almaktadır.

4. UYGULAMA VE GERÇEKLEŞTİRİM

Bu bölümde, geliştirilen yapay zeka tabanlı anomali tespit sisteminin uygulama aşamaları, yazılım mimarisi, kullanılan teknolojilerin detayları ve kullanıcı arayüzü tasarımı açıklanmaktadır.

Projenin geliştiriminde Agile bir yapı kullanılmıştır. Süreç, planlama, geliştirme, test etme şeklinde ilerlemiştir. Projenin temel programlama dili olarak Python (v3.9) seçilmiştir. Geliştirme ortamı olarak Pycharm kullanılmıştır. Pylance ve Python eklentileri ile kod tamamlama, hata ayıklama (debugging) ve linting özelliklerinden yararlanılmıştır. Hızlı denemeler ve görselleştirmeler için Jupyter Notebook da kullanılmıştır. Model ve ön işleme pipeline'larının diske kaydedilmesi ve yüklenmesi için joblib ve pickle kütüphaneleri kullanılmıştır. Bu projede iki farklı birbirleriyle bütünleşmiş bir şekilde çalışan ana bileşen kullanılmıştır. Hem model eğitimi hem de kullanıcı arayüz kısmı birbirlerinden ayrılarak projenin geliştirilme kısmı kolaylaştırılmıştır.

4.1 Kod Yapısı

- **main.py:** Model eğitimini, değerlendirmesini ve model dosyalarının kaydedilmesini içeren sınıftır. Öncelikle ilk programlanan sınıf budur. Bu sınıf öncelikle veri işleme, makine öğrenimi modellemesi ve dosya yönetimi için gerekli kütüphaneleri yükler. Bu süreçte pandas ve numpy veri işleme ve analizi için, re (regex) ise log satırlarından anlamlı kısımları çıkartmak için (blok ID'leri, IP adresleri vb.) kullanılmıştır. Ardından, modelin log verilerini anlayabilmesi için veri işleme ve özellik çıkarımı yapılmıştır. İlk olarak “get_hdfs_optimized_stopwords()” fonksiyonu HDFS loglarında bazı gereksiz terimleri içeren ve o terimleri log dosyalarından çıkaran kelime listesini oluşturarak ön işlemeyi daha doğru ve hızlı olarak yapılmasını sağlamıştır. Bu adım günümüz şartlarında donanımsal olarak 16 GB gibi düşük RAM’li cihazlar için çok önemlidir. Sonrasında, “parse_log_and_extract_features()” fonksiyonu ise ham log satırlarını satır satır okur ve her bir log kısmı için “HasException, HasTimeout” gibi işlemler yapar. Bu işlemler, öncelikle log satırında 'exception' kelimesi geçiyor mu geçmiyor mu, Log satırında 'timeout' kelimesi geçiyor mu geçmiyor mu, Log satırında 'connection' kelimesi geçiyor mu geçmiyor mu, Log satırında 'replication' kelimesi geçiyor mu geçmiyor mu şeklinde kontroller yapar. Bu yapılar, modelin anomali olabilecek durumları tespit etmesine yardım eder. Sayısal (numeric) özellikler için ise “UniqueWords, ContentLength” yapıları kullanılmıştır. UniqueWords, log bloğundaki (aynı BlockId'ye ait tüm satırlar) benzersiz kelime sayısını gösterir. Eğer yüksek bir değer çıkarsa bu durum anomali olabileceği anlamına gelmektedir. ContentLength, Bir log bloğundaki metnin toplam karakter sayısını gösterir. Anormal derecede uzun veya kısa loglar, bir anomali olabilir.
- Özellik çıkarma işlemi tamamlandığından sonra veri seti anomali tespiti için hazır hale gelir. Veri seti, modelin öğrenmesi için kullanılan eğitim seti ve performans değerlendirilmesi için kullanılan test setine ayrıştırılmıştır. Daha sonraki adımda projenin en önemli kısmı olan XGBoost, LightGBM ve CatBoost gibi gradient boosting algoritmaları ile eğitime başlanmıştır. Daha sonrasında geri kalan modeller olan KNN ve RandomForest modelleri öncelikle tek başlarına eğitilmiştir. Daha sonrasında ise “sklearn.ensemble.VotingClassifier” kullanılarak Hibrit model oluşturulmuştur. Ayrıca, hibrit yapıda “voting='soft'” olarak alınmıştır. Bu

parametre, modellerin sınıf tahminleri yerine olasılık tahminleri (predict_proba) üzerinden oy kullanacağını belirtir. Bu, daha incelikli bir karar alma süreci sağlar. Örneğin, bir model bir veri noktasını %51 olasılıkla anomali, diğeri ise %90 olasılıkla anomali olarak tahmin ediyorsa, soft oylama bu farklılıkları dikkate alır. Bu durum da doğruluk oranı daha yüksek sonuçlar elde edilmiştir. Tahmin işlemleri yapılmasının ardından performansı fl_score, accuracy ve classification_report gibi yapılarla değerlendirilmiştir. Test ve performans kısmından 5. Kısımda bahsedilmektedir.

Son olarak eğitim ve modelleme aşamalarından geçen modeller, kaydedilmiştir. Kayıt aşaması, joblib kütüphanesi aracılığıyla gerçekleştirilmiştir. joblib.dump() fonksiyonu kullanılarak, her bir eğitilmiş model. joblib uzantılı bir dosya olarak outputs klasörüne kaydedilmiştir. Bu yöntem, GUI uygulamasının her çalıştığında modelleri tekrar eğitmek zorunda kalmadan, doğrudan yükleyip kullanılabilmesini sağlar. Böylece, main.py dosyası, uygulamanın hızlı ve verimli bir şekilde çalışabilmesi için gerekli olan tüm temel hazırlık ve altyapı çalışmalarını yapmış oldu.

- **AnomalyDetectionGUI.py:** Bu sınıf, projenin kullanıcı için ayrılmış kısmıdır. Tkinter kütüphanesi kullanılarak oluşturulan bir arayüzü barındırır. Bu dosya, main.py tarafından eğitilen modellerin yüklenmesini ve log dosyaları üzerinde anomali tespiti yapılmasını sağlar.

Sınıf, arayüzün tüm görsel bileşenleri, veri işleme, dosya yönetimi ve görselleştirme için gerekli kütüphaneleri en başta yükler. Bu kütüphaneler “Tkinter, Pandas, NumPy, Joblib, Matplotlib, ReportLab” kütüphaneleridir. Tkinter, arayüz (GUI) bileşenlerini oluşturmak için kullanılır. Pandas ve NumPy, log verilerinin işlenmesi ve sayısal işlemler için kullanılır. Joblib, main.py tarafından kaydedilen makine öğrenimi modellerini yüklemek için kullanılır. Matplotlib, Analiz sonuçlarını (örneğin, anomali dağılımı) görselleştirmek için kullanılır. ReportLab, PDF raporları oluşturmak için kullanılır.

AnomalyDetectionGUI.py, main.py'deki gibi bir log ayrıştırma fonksiyonu içerir. “FlexibleLogParser” adlı metot sayesinde farklı HDFS log formatlarını otomatik olarak algılar ve işler. Bu projenin en önemli özelliklerinden biridir. Çünkü kullanıcı 40 öznitelik altında ya da üstünde bir ham veri setine sahipse bile anomali tahmin ve etiketleme işlemi yapılabilir. Proje için büyük bir esneklik sağlar. Bunu

“dummy(sahte)” özellikler ile yapar. Eğer çıkarılan özellik sayısı 40 özellikten az ise eksik olan verilerin yerine dummy(sahte) veriler atanır. Anomali tespiti sırasında doğruluk oranına etkisi olmaması için tüm değerleri “0” olarak atanır ve sistem anomali tespiti için dummy (sahte) verileri esas almaz. Eğer log dosyasından beklenenden daha fazla özellik çıkarılırsa, modelin tanımadığı bu ekstra özellikler tahmin yapmadan önce veri setinden otomatik olarak kaldırılır.

Projenin tüm arayüz mantığı, bu sınıf içinde yer alır. “__init__ (self, root)” Sınıfın kurucu (constructor) metodudur. Uygulama başlatıldığında ilk çalışan kısımdır. “root” tkinter'in ana kısmıdır. Uygulamanın başlığını (Anomali Tespit Sistemi) ve boyutlarını ayarlar. Gerekli değişkenleri (current_model, hdfs_file_path, analysis_results gibi) başlatır. “Setup_ui()” fonksiyonunu çağırarak tüm arayüz bileşenlerini oluşturur. Load_model()” fonksiyonunu çağırarak başlangıçta varsayılan bir modeli (model_xgb_regularized.joblib) otomatik olarak yükler. Kullanıcı “Görsel 1”da görüldüğü üzere sol panelden modeli istediği gibi seçebilir. Arayüzün kullanıcı eylemlerini yöneten fonksiyonlar içerir. “load_model()” fonksiyonu, “outputs” klasörüne, önceden main.py de eğitim kaydettiğimiz “.joblib” türlü model dosyasını yükler. “load_log_file()” Kullanıcıya bir dosya seçme penceresi açar ve seçilen HDFS log dosyasının yolunu kaydeder. Diğer bir önemli fonksiyon “run_analysis()” kısmıdır. Görsel 1.1’ de görüldüğü üzere HDFS ham verilerinin ve modelin seçiminin ardından analizi başlatma seçeneğine kullanıcı tıkladığında “run_analysis()” fonksiyonu devreye girer. Bu fonksiyon arka plandaki sonuç hesaplama, anomali tahmini yapma gibi işlemleri yapar. Analiz tamamlandığında, sonuçlar arayüzün sağ tarafındaki panele yerleştirilir. “save_pdf() ve save_excel()” fonksiyonları ise analiz sonuçlarını detaylı bir şekilde kaydeder. Görsel 5’ ve 1,5 bu raporlamaya örnektir.

- **_1_imports_and_setup.py:** Bu sınıf, projenin başlangıç yapısını oluşturan ana kısımlardan biridir. Bu sınıf gerekli kütüphaneleri yüklemenin ardından sistemin düzenli çalışıp çalışmadığını test eder. Bu sınıf çalıştırıldığında 3 adet kişiye seçenek sunar.

Seenekler:

1. Sistem testini alıřtır
2. GUI'yi direkt bařlat
3. Test + GUI

|

Seiminizi yapın (1/2/3) [varsayılan: 2]:

Görsel 7 Seenekler.

Class alıřtırıldığında görsel 7’de görüldüğü üzere 3 adet seenek sunar. Bu seenekler arasından kullanıcı “1. Sistem testini alıřtırı seerse”, HDFS Processing Test bařlatılır ve HDFS fonksiyonları mevcut mu, Dummy veri oluřturma alıřıyor mu, HDFS processing bařarılı mı řeklinde testler yapar. Test sonucunda eğer alıřmayan bir kısım varsa bildirilir ardından “model Dizini Kontrolü” yapılır. Bu kontrolde model dosyaları joblib formatında bulunuyor mu kontrol edilir. Toplam 8 adet dosyanın bulunması gerekir. Ardından, kütüphane kontrollerini yapar. Pandas, numpy, Matplotlib, Tkinter kütüphanelerini kontrol eder. Eğer tüm kontroller bařarıyla gerekleřtirilirse görsel 8 de olduđu gibi bařarılı olduđuna dair bir geri dönüş alınır.

```
Seçiminizi yapın (1/2/3) [varsayılan: 2]: 1
Sistem testi başlatılıyor...
```

1. HDFS Processing Test:

- ✓ HDFS fonksiyonları mevcut
- ✓ Dummy veri oluşturma çalışıyor
- ✓ HDFS processing başarılı

2. Model Dizini Kontrolü:

- ✓ 8 model dosyası bulundu:
 - label_encoder.joblib
 - model_catboost.joblib
 - model_hybrid.joblib
 - model_knn.joblib
 - model_lightgbm.joblib
 - model_rf.joblib
 - model_xgb_regularized.joblib
 - scaler.joblib

3. Kütüphane Kontrolü:

- ✓ pandas - 2.2.3
- ✓ numpy - 2.2.0
- ✓ matplotlib - OK
- ✓ tkinter - OK

Görsel 8 Başarılı Test Aşaması Görseli.

Eğer 2. Seçenek olan “GUI’yi direkt başlat” seçeneği seçilirse arayüz direkt başlatılır ve görsel 1 da bulunan arayüz açılır. 3.seçenek seçildiği durumda test aşaması yapılır görsel 8’de bulunan başarılı test aşaması ardından arayüz görsel 1’da bulunan arayüz açılır.

- **2_hdfs_processing.py**: Sınıfı, projede ham HDFS log verilerini işlenebilir bir formatta dönüştüren ve makine öğrenimi modelleri için anlamlı özellikler çıkaran yardımcı bir modüldür. Bu modül, ön işleme aşamasını gerçekleştirmektedir. İlk aşama olarak veri işleme ve düzenli ifade (regex) operasyonları için gerekli

kütüphaneleri yükler. Re, numpy ,pandas kütüphanelerini kullanır. Re log satırlarındaki belirli yapıları (örneğin blok ID) çıkartmak için kullanılır. Numpy ve Pandas sayısal işlemler ve veriyi DataFrame yapısında düzenlemek için kullanılır. En önemli fonksiyonu “parse_hdfs_log” fonksiyonudur. Bu fonksiyon satır satır okuma ve parçalama işlemi yapar. Örneğin “re.search(r'blk_(-?\d+)', line)” düzenli ifadesi, "blk_" ile başlayan ve ardından sayısal bir değer gelen metinleri arar. Bu, her bir log satırını belirli bir blokla ilişkilendirir. Ayırıştırılan her log satırı, LineNumber, BlockId ve LogContent gibi bilgileri içeren bir sözlük olarak kaydedilir ve sonunda bir Pandas DataFrame'e dönüştürülür. Bu yapı, veriyi daha kolay analiz edilebilir hale getirir. “extract_features_for_block” fonksiyonu bir başka önemli fonksiyonlarından biridir. Bu fonksiyon aynı BlockId'ye sahip tüm log satırlarını bir araya getirir ve anomali tespiti için gerekli olan özellikleri çıkarır. Bu fonksiyon ayrıca birleştirilmiş metin içinde 'exception', 'timeout', 'connection' ve 'replication' gibi anahtar kelimelerin geçip geçmediği kontrol eder. Eğer bir kelime bulunursa ilgili özellik 1 olarak işaretlenir, eğer bulunmazsa 0 olur. Bu özellik, sistem hatalarını ya da olağan dışı durumları tespit etmede işe yarar. Ayrıca, Metnin uzunluğu (ContentLength) ve metindeki benzersiz kelime sayısı (UniqueWords) gibi sayısal özellikler de bu fonksiyonda hesaplanır. Ayrıca bu sınıfta kendi modülleri doğru çalışıyor mu bunu kontrol etmek için “test_hdfs_processing” fonksiyonu bulunmaktadır. Bu fonksiyon, modülün kendi içinde doğru çalışıp çalışmadığını kontrol etmek için yazılmıştır. Log ayırıştırma ve özellik çıkarma işlemlerini sahte veriler üzerinde test eder ve çıktıları konsola yazdırır.

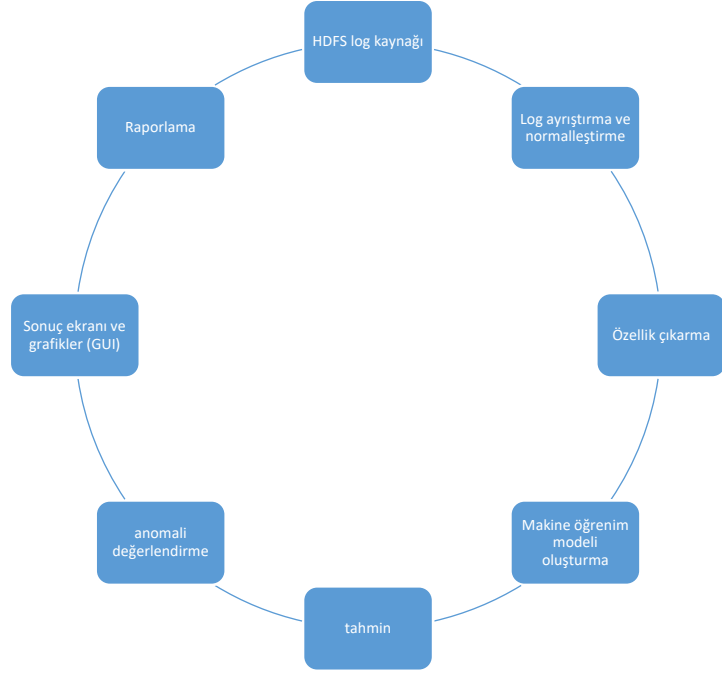
- **_3_gui_main_class.py:** Bu sınıf, temel arayüz işlemlerini bir araya getiren kısımdır.
- **_4_gui_panels.py:** Bu sınıf, projenin görsek düzenini oluşturan kısımdır. Sol panel ve sağ panel olmak üzere iki kısım bulunur. Görsel 1’ de bu arayüz yapısını görebilirsiniz. Sol Panel (setup_left_panel), Kullanıcıya önceden eğitilmiş modeller (XGBoost, CatBoost, LightGBM, Random Forest, Hibrit, KNN) arasından seçim yapma imkânı sunan bir “LabelFrame” yer alır. Bu sayede kullanıcı, farklı modellerin log verileri üzerindeki performansını kolayca karşılaştırabilir. Sol panelde ayrıca log dosyalarını yüklemek için panel bulunur. Analiz butonu analiz işlemini seçim işlemi ardından başlatır. Rapor butonları ile raporlanabilir. Sağ Panel (setup_right_panel), kullanıcının analiz sonuçlarını görebileceği ve

değerlendirebileceği alandır “3.1.8.1 Uygulama Giriş Bölümü” kısmında detaylı olarak sağ ve sol panel anlatılmıştır.

- **_5_gui_visualization.py:** Genel olarak görselleştirmeler için oluşturulmuş sınıftır. Dosya, arayüz ve görselleştirme için gerekli kütüphaneleri içe aktarır (Tkinter, ttk, Matplotlib, Datetime). Tkinter ve ttk ana arayüzü ve sekme kısmı oluşturmak için kullanılır. Matplotlib veri görselleştirmesi için kullanılır. “FigureCanvasTkAgg” sınıfı, Matplotlib grafiklerini doğrudan Tkinter arayüzüne de göstermeyi sağlar. Datetime, Anlık zamanı gösteren durum çubuğundaki tarih ve saat bilgisini sağlamak için kullanılan kütüphanedir. Yani aslında temel amacı ana pencereye sekme yapısını eklemek ve arayüzü daha düzenli hale getirmektir. “__init__(self, root)” sınıfın kurucu metodu, Tkinter penceresini ayarlar ve ttk Notebook yapısını oluşturur. `setup_visualization_tab()` “Görselleştirme” başlıklı bir sekme oluşturur ve bu sekmeye Matplotlib grafiği için bir alan ekler. Figure ve FigureCanvasTkAgg sınıfları, grafiklerin bu alanda çizilmesini sağlar. “`setup_detailed_analysis_tab()`”, “Detaylı Analiz” kısmını oluşturur. Bu sekme, sonuçların daha ayrıntılı bir şekilde (bir tablo içinde) gösterilmesini sağlar. “`setup_status_bar()`” uygulamanın alt kısmında, kullanıcının o anki durumu (örneğin, “Model yüklendi”) ve mevcut saati görebileceği bir durum çubuğu oluşturur. Kısacası bu modül projenin analiz yapmasını ve kullanıcılar için anlaşılır bir şekilde bu analizi sunmasını sağlar.
- **_6_gui_model_functions.py:** Uygulamanın en önemli görevlerinden birini, yani main.py tarafından eğitilen makine öğrenimi modellerini yükleme ve yönetmeyi sağlar. Bu dosya kullanıcı arayüzü ve modeller arasındaki bağlantıyı sağlar. Yani aslında bu yapı uygulamanın doğruluğunu tamimiyle etkiler.
- **_7_gui_analysis_function.py:** Gerçek anomali tespiti (dummy kısımlarını değil) yapıldığı kısımdır. Kullanıcı arayüzünün tetiklediği analiz sürecini yönetir ve main.py tarafından eğitilmiş modelleri kullanarak log verileri üzerinde tahminler yapar. Yani “_7_gui_analysis_function.py” dosyası, kullanıcıdan alınan girdileri (log_file), uygulamanın çekirdek veri işleme ve modelleme mantığıyla birleştirerek, anlamlı ve raporlanabilir anomali tespiti sonuçları üretir. Bu modül, uygulamanın ön yüzü (AnomalyDetectionGUI.py) ile arka plan veri analizi (main.py) arasında bir köprü görevi görerek tüm sürecin uyumlu bir şekilde çalışmasını sağlar.

- **_8_gui_display_results.py:** Görsel olarak göze hitap edecek şekilde analiz sonucu gözüksün diye oluşturulmuş bir yapıdır. Bu modül, sonuçların kişiye sunulması için önemlidir çünkü görsellik kullanıcının memnuniyetini artırır. İlk aşama olarak kütüphaneleri içeri aktarır. “tkinter ve ttk” metin kutusu tablo oluşturma gibi yapılar için kullanılır. “Matplotlib.pyplot” grafik çizimi için kullanılır. “matplotlib.backends.backend_tkagg” Matplotlib grafiklerini Tkinter arayüzüne yerleştirmeyi sağlar. En önemli fonksiyonu “display_results(analysis_results, output_text)” fonksiyonudur. Bu fonksiyon, analizin temel özetini arayüzdeki metin kutusuna yazar. Metnin kutusunun içeriğini temizler ve Analiz sonuçları sözlüğünden (dictionary) alınan bilgileri kullanarak, toplam blok sayısı, normal ve anomali blok sayıları ile anomali yüzdesi gibi temel istatistikleri metin olarak kullanıcıya sunar. Bir başka fonksiyonu “plot_distribution(analysis_results, plot_frame)”, bu fonksiyon Normal ve anomali olarak tahmin edilen log bloklarının sayısını kullanarak bir çubuk grafik (bar chart) oluşturur. Görsel 3 de oluşturulan çubuk grafik (bar chart) görülebilir.
- **_9_gui_report_functions.py:** Bu yapı projenizin analiz sonuçlarını kalıcı olarak saklamak ve paylaşmak için raporlar oluşturan bir modüldür. Görsel 5 ve görsel 6’te görüldüğü üzere proje PDF ve Excel formatında raporlama yapar. “AnomalyDetectionGUI.py” tarafından çağrılır.
“generate_pdf_report(analysis_results,filename="anomaly_report.pdf")”
fonksiyonu en önemli iki fonksiyonundan biridir. Bu fonksiyon analiz sonuçlarını derli toplu bir PDF belgesine dönüştürür. Bu süreçte ReportLab kütüphanesini kullanır. Bir diğer önemli fonksiyonu “generate_excel_report(analysis_results, filename="anomaly_report.xlsx”)” fonksiyonu, analiz sonuçlarının daha ayrıntılı ve işlenebilir bir versiyonunu Excel dosyası olarak kaydeder. Bu süreçte Pandas kütüphanesi kullanılır.
- **_10_run_application.py:** Direkt GUI yi başlatmayı yarayan ana merkezi yapıdır. En önemli fonksiyonu “run_anomaly_detection_gui()” fonksiyonudur. Bu fonksiyon, uygulamanın kullanıcı arayüzünü başlatmaktan sorumludur.

4.2 Proje geliştirme döngüsü



Şekil 1 Proje geliştirme döngüsü.

Şekil 1 de görüldüğü üzere öncelikle HDFS Log Kaynağı elde edilmiştir.

- 1) **HDFS Log Kaynağı:**
Sistemin analiz edeceği ham verilerin kaynağı. Bu, HDFS (Hadoop Dağıtılmış Dosya Sistemi) sistemlerinden gelen, genellikle karmaşık ve yapılandırılmamış metin dosyalarıdır.
- 2) **Log Ayrıştırma ve Normalleştirme:**
Ham log verisini, modelin işleyebileceği bir formata dönüştürme aşamasıdır.
- 3) **Özellik Çıkarma (Feature Extraction):** Normalleştirilmiş log verisinden, anormali tespiti için gerekli olan anlamlı özelliklerin oluşturulduğu aşama. Örnek olarak, bir log bloğu içinde exception veya timeout kelimelerinin varlığı gibi ikili (boolean) özellikler ve kelime sayısı gibi sayısal özellikler çıkarılır.
- 4) **Makine Öğrenimi Modeli:** Çıkarılan özellik setini kullanarak anormali tespiti yapan ana algoritma seçimi ve eğitimi.
- 5) **Tahmin (Prediction):** Makine öğrenimi modelinin, her bir log bloğunu "Normal" veya "Anormali" olarak sınıflandırdığı çıktılar.
- 6) **Anormali Değerlendirme:** Model tarafından anormali olarak etiketlenen log bloklarının, daha sonra incelenmek üzere ayrıldığı aşama. Bu, yanlış pozitifleri azaltmak ve gerçek anormalilere odaklanmak için önemlidir.
- 7) **Sonuç Ekranı ve Grafikler (GUI):** Analiz sonuçlarının, grafiksel kullanıcı arayüzü (GUI) üzerinde görsel olarak sunulduğu kısım. Burada, anormali sayısı, oranı ve anormali dağılımı gibi bilgiler yer alır.
- 8) **Raporlama (PDF/Excel):** Analiz sonuçlarının, kullanıcı tarafından kolayca paylaşılabilmesi ya da kaydedilebilmesi için PDF veya Excel gibi formatlarda raporlandığı son aşama.

5. BULGULAR VE TARTIŞMA (TESTLER VE SONUÇLAR)

Bu bölümde, araştırmamızda elde edilen bulgular, alt bölümler, düz yazı, sayı, şekil ve tablolar halinde sunulmuştur. Elde edilen bu yeni bulgular hem kendi içinde hem de daha önce bilinen yöntemlerle karşılaştırılarak tartışılmış, bilime ve uygulamaya olan katkısı ortaya konulmuştur. Böylece, "bu araştırmada yeni olarak neler bulundu ve yeni bulguların önemi, değeri, bilime ve uygulamaya katkısı nedir?" sorularına cevap verilmiştir.

5.1 Uygulamanın Test Senaryoları ve Doğrulama Adımları

Anomali tespit sistemimizin performansı, veri setinin %70'i ile eğitilip, geri kalan %30'u ile test edilerek doğrulanmıştır. Ayrıca farklı veri setleri üzerinde de doğrulama işlemi yapılmıştır. Bu Yaklaşım, modellerin daha önce hiç görmediği veriler üzerindeki genelleme yeteneğini ölçmemizi sağlamıştır. Uygulama, log dosyasını yükleme, model seçimi, analiz başlatma ve sonuçları PDF veya Excel formatında raporlama gibi temel işlevleri başarıyla yerine getirmiştir. Bu testler, sistemin stabil ve pratik kullanım için hazır olduğunu teyit etmiştir.

5.2 Performans Ölçümleri, Hata Oranları ve Başarı Yüzdeleri

Anomali tespiti modelimizin performansı, doğruluk, F1-skoru, hassasiyet (precision) ve duyarlılık (recall) gibi standart metrikler kullanılarak değerlendirilmiştir. Bulgularımız, tüm modellerin anomali tespiti görevinde son derece yüksek başarı oranlarına ulaştığını göstermektedir. Özellikle, güvenlik odaklı sistemlerde kritik bir metrik olan anomali tespiti duyarlılığına (recall) ve F1-skoruna odaklanılmıştır.

MODEL PERFORMANS METRİKLERİ KARŞILAŞTIRMASI

Model	Accuracy	F1-Score	F1-Anomaly	Precision	Recall	ROC-AUC	Training_Time(s)
Hybrid	0.9999	0.9999	0.9979	1.0000	0.9999	1.0000	34.94s
KNN	0.9998	0.9999	0.9973	1.0000	0.9999	0.9998	0.12s
XGBoost	0.9998	0.9999	0.9973	0.9999	0.9999	1.0000	3.47s
LightGBM	0.9998	0.9999	0.9969	1.0000	0.9999	1.0000	1.84s
CatBoost	0.9998	0.9999	0.9967	1.0000	0.9998	1.0000	5.42s
RandomForest	0.9996	0.9998	0.9932	0.9998	0.9998	0.9999	10.43s

Görsel 9 Model Performans metrikleri tablosu

Görsel 9’da görüldüğü üzere modeller gayet iyi performanslar göstermiştir. Özellikle anomali sınıfına ait F1-Skoru, Duyarlılık (Recall) ve Hassasiyet (Precision) değerleri, modellerin gerçek anomali tespitindeki başarısını göstermektedir. Tabloda görüldüğü kadarıyla da Hibrit modelin en iyi performansı gösterdiği görülmüştür. Bu beklenmedik sonucun temel nedeni KNN in yapısı gereği düşük loglarda iyi performans göstermesi ve RandomForest modelinin yüksek loglarda iyi performans göstererek birbirlerinin eksiklerini kapatmalarıdır. Sistemin güvenilirliği açısından en kritik metrik olan duyarlılık, tüm gerçek anomalilerin ne kadarının başarıyla tespit edildiğini gösterir. Hibrit model, bu kategoride 0.9991 ile diğer tüm modellere göre üstün bir performans göstermiş, bu da çok az sayıda anomalinin gözden kaçırıldığını kanıtlamıştır. Tüm modellerin ROC-AUC değerlerinin 0.9998’in üzerinde olması, sınıflandırma yeteneklerinin iyi olduğunu göstermektedir.

Tablodaki veriler, tekil modellere göre hibrit modelin daha yüksek performans sağladığını ve bu nedenle projenin temel hedeflerinden biri olan güvenilir ve yüksek performanslı bir anomali tespit sistemi oluşturma amacına ulaşıldığını kanıtlamaktadır. Bu bulgu, özellikle güvenlik ve sistem izleme gibi alanlarda anomali tespitinin etkinliğini artırmak için hibrit modelleme yaklaşımının önemini vurgulamaktadır.

Bu veriler veri setinin %30’luk test setine bölünmesinin ardından test veri seti üzerinden elde edilmiştir.

CONFUSION MATRIX DEĞERLERİ

Model	TP	FP	FN	TN
Hybrid	111633	2	12	3366
KNN	111632	5	13	3363
XGBoost	111636	9	9	3359
LightGBM	111629	5	16	3363
CatBoost	111627	4	18	3364
RandomForest	111621	22	24	3346

Görsel 10 Confusion Matrix Değerleri

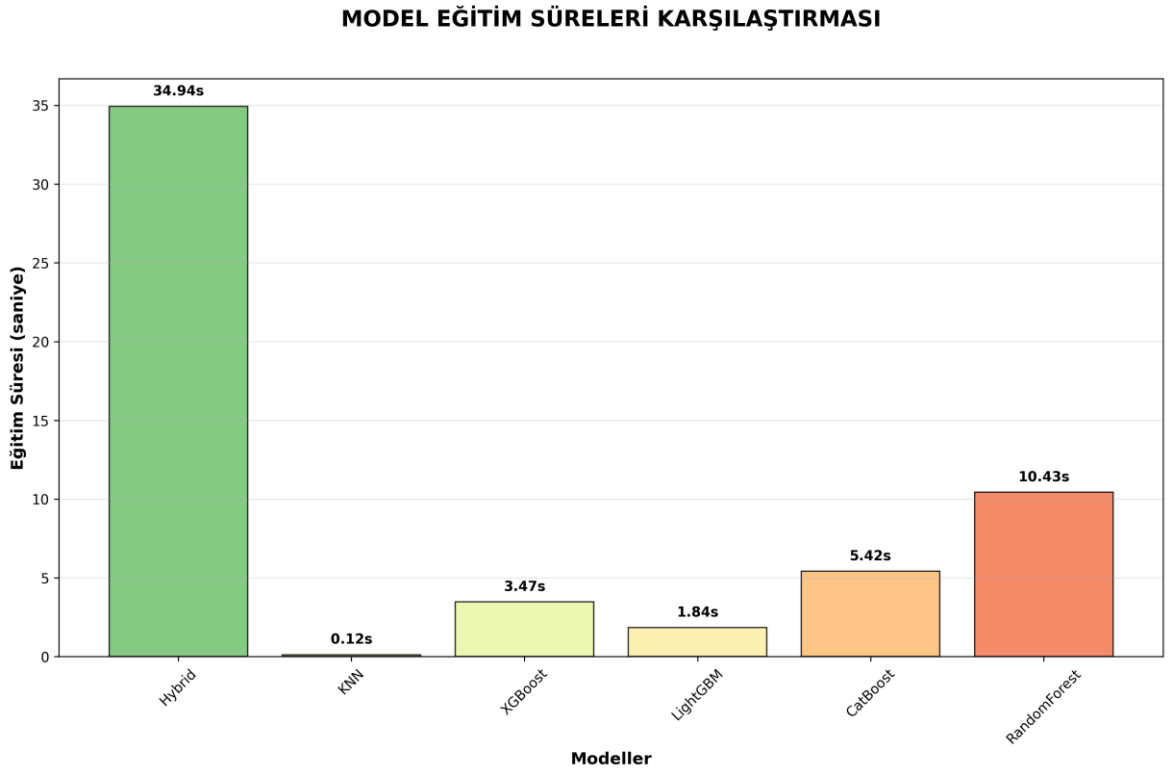
Görsel 10'da görüldüğü üzere anomali tespiti için eğitilen altı farklı makine öğrenimi modelinin, test veri seti üzerinde detaylı performans karşılaştırılması görülmektedir.

Tablodaki matrixler şunlardır:

- **TP (True Positive - Gerçek Pozitif):** Doğru şekilde anomali olarak tespit edilen log sayısı.
- **FP (False Positive - Yanlış Pozitif):** Normal olmasına rağmen yanlışlıkla anomali olarak etiketlenen log sayısı. Yanlış alarmları temsil eder.
- **FN (False Negative - Yanlış Negatif):** Anomali olmasına rağmen yanlışlıkla normal olarak etiketlenen log sayısı. Sistem güvenliği açısından en kritik hata türüdür, çünkü gerçek bir tehdidin gözden kaçtığı anlamına gelir.
- **TN (True Negative - Gerçek Negatif):** Doğru şekilde normal olarak tespit edilen log sayısı.

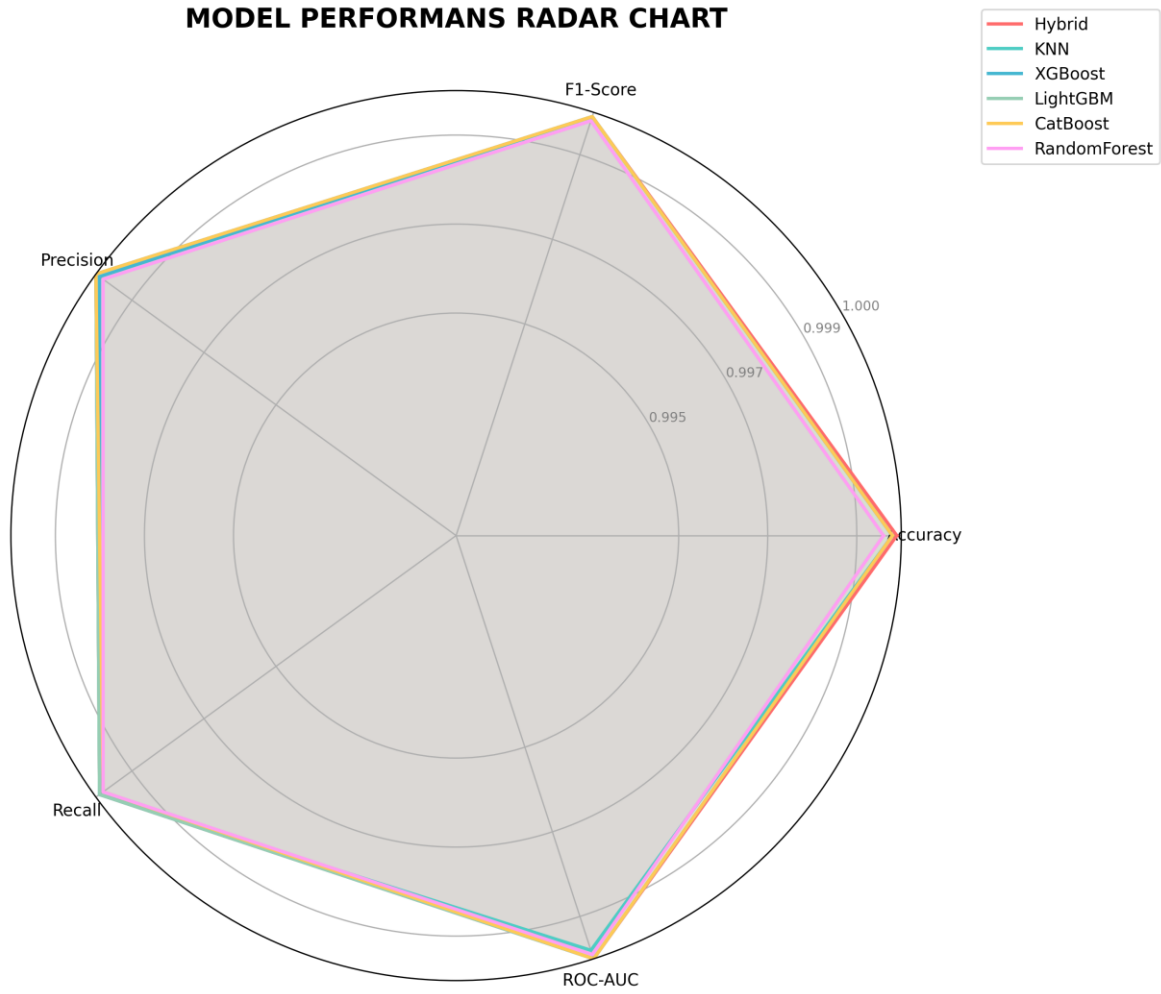
Hibrit Model, 12 FN değeri ile en düşük yanlış negatif oranına sahip modellerden biridir (sadece XGBoost daha düşük bir FN'ye sahiptir ancak Hibrit Model'in diğer metrikleri daha yüksektir). Bu durum, hibrit yaklaşımın gerçek anomali tespiti konusunda en güvenilir performansı sunduğunu kanıtlamaktadır. Güvenlik ve sistem sağlığı izlemesi gibi alanlarda, bir anomalinin gözden kaçması (FN) yanlış bir alarmdan (FP) çok daha

ciddi problemler oluşabilir. Yanlış Pozitif (FP) sayıları da benzer şekilde düşük değerlerdedir. Bu, sistemin gereksiz alarmlar üreterek kullanıcıyı meşgul etmediğini ve operasyonel verimliliği bozmadığını göstermektedir. RandomForest, 24 FN ile en fazla anomaliyi gözden kaçıran model olmuştur. Bu, bu modelin tek başına kullanıldığında anomali tespiti için daha az güvenilir olduğunu işaret etmektedir. Bu modellerin hepsi kullanıcıya sunulmuştur. Kullanıcı istediği modeli seçerek doğruluk oranının en yüksek çalıştığını düşündüğü modeli kullanmalıdır.



Görsel 10 Model Eğitim Süreleri Karşılaştırılması

Görsel 10’da görüldüğü üzere “Hibrit” modelin eğitimi çok uzun sürmüştür. Performans kriterlerinden biri olan bu durum hibrit model için olumsuz bir tablodur. Ancak tek başına modelin kötü olduğunu söyleyebilecek bir nitelikte olan kriter değildir.



Görsel 11 Model Performans Radar Chart'ı

Görsel 11'de bulunan Chart'ta anomali tespiti için en kritik olan altı performans metriğindeki başarısını görsel olarak karşılaştırmaktadır. Grafikteki her bir eksen bir metriği temsil eder ve modelin eksenden ne kadar uzakta olduğu, o metriktaki başarısını gösterir. Grafiğin en belirgin bulgusu, Hibrit Modelin (mavi çizgi) diğer tüm modellere göre daha geniş bir alanı kaplaması ve neredeyse her metrikte en yüksek skorlara ulaşmasıdır. Bu durum, Hibrit modelin sadece tek bir metrikte değil, aynı zamanda dengeli ve üstün bir genel performans sunduğunu göstermektedir. Özellikle, Anomali F1-Skoru, Anomali Precision ve Anomali Recall eksenlerinde Hibrit modelin diğer çizgilere göre daha dışta yer alması, anomali sınıfını tespit etme konusunda diğer modellerden daha başarılı olduğunu göstermektedir.

RandomForest:
Train Accuracy: 0.999678
Test Accuracy: 0.999600
Accuracy Farkı: 0.000078 (0.01%)
Train F1-Anomaly: 0.994503
Test F1-Anomaly: 0.993173
F1 Farkı: 0.001330 (0.13%)
Durum: ☒ İYİ GENELLEME
XGBoost:
Train Accuracy: 0.999887
Test Accuracy: 0.999843
Accuracy Farkı: 0.000043 (0.00%)
Train F1-Anomaly: 0.998068
Test F1-Anomaly: 0.997328
F1 Farkı: 0.000741 (0.07%)
Durum: ☒ İYİ GENELLEME
LightGBM:
Train Accuracy: 0.999937
Test Accuracy: 0.999817
Accuracy Farkı: 0.000120 (0.01%)
Train F1-Anomaly: 0.998925
Test F1-Anomaly: 0.996888
F1 Farkı: 0.002037 (0.20%)
Durum: ☒ İYİ GENELLEME
CatBoost:
Train Accuracy: 0.999880
Test Accuracy: 0.999809
Accuracy Farkı: 0.000072 (0.01%)
Train F1-Anomaly: 0.997961

Test F1-Anomaly: 0.996741
F1 Farkı: 0.001221 (0.12%)
Durum: ☒ İYİ GENELLEME

Tablo 1 Aşırı öğrenme (Overfitting) Analizi Tablosu

Tablo 1’de görüldüğü üzere tüm modeller iyi genelleme yeteneğine sahiptir. Bu durum bütün modellerin aşırı öğrenmeden uzak bir yapıda olduğunu göstermektedir. Yani modeller ezber yapmamışlar verileri öğrenmişlerdir. Makine öğreniminde bir modelin iyi genelleme yapabilmesi, yani daha önce hiç görmediği veriler üzerinde de yüksek performans sergileyebilmesi önemlidir. "İyi Genelleme" durumu, eğitim ve test performans metrikleri arasındaki farkın çok küçük olması mantığına dayanır. En yüksek genellemeye sahip olan LightGBM modelinde %0,20 olarak gözlemlenmiştir, ancak bu değer bile bir aşırı öğrenme göstergesi için yok denecek kadar küçüktür.

Model / Çalışma	F1-Score (Genel)	F1-Score (Anomaly)
Bu Proje (Hibrit olan esas alınmıştır.)	0.99	0.9978
CLDTLog [28]	0.9971	0.9940
LogBERT [9]	0.8232	0.7100
LogLead [15]	0.9500	0.9200
LogRobust [29]	0.9900	0.9800
DeepLog[28]	0.9800	0.9600

Tablo 2 Aynı Veri Setini Kullanan Proje Karşılaştırma Tablosu.

Tablo 2’de görüldüğü üzere projede kullanılan Hibrit yaklaşım diğer projelere kıyasla çok daha yüksek F1 skorlarına sahiptir. Bu başarının temel nedeni, projede kullanılan hibrit yaklaşımın metin tabanlı verilerin sayısallaştırılması ile sağladığı güçlü özellik mühendisliği sayesinde, KNN ve Random Forest'in tamamlayıcı güçlerini en iyi şekilde kullanmasından kaynaklanmaktadır. Bu yaklaşım, sadece metin verilerini anlamlı sayısal verilere dönüştürmekle kalmamış, aynı zamanda KNN'in aykırı değer tespiti ve Random Forest'in karmaşık ilişkileri yakalama yeteneklerini birleştirerek hem basit hem de karmaşık anomali

türlerini yüksek bir doğrulukla tespit etmeyi başarmıştır. Bu sinerji, diğer modellerin elde ettiği sonuçlara kıyasla daha yüksek ve tutarlı F1 skorları elde etmemizin olmuştur.

Aynı veri seti kullanılarak yapılan çalışmalar incelendiğinde, HDFS1 veri seti üzerinde çeşitli yaklaşımlarla farklı doğruluk oranları elde edildiği görülmektedir. Örneğin, Loglizer [14] aracı kullanılarak uygulanan Invariants Mining yöntemi %87,2, PCA tabanlı yöntem %95,1 ve Random Forest algoritması %96,8 doğruluk oranlarına ulaşmıştır. Derin öğrenme tabanlı bir yaklaşım olan DeepLog[28], HDFS1 veri seti üzerinde %96,9 F1-skor elde ederken, bir diğer ileri seviye model LogBERT [9] %98,4 gibi oldukça yüksek bir performans sergilemiştir. WraySmith/log-anomaly [16] projesi, CNN tabanlı modeliyle %99'un üzerinde F-skor rapor etmiştir. LogLead çalışmasının karşılaştırmalı analizinde ise Isolation Forest %97,1, LSTM %98,3 ve Logistic Regression %95,6 doğruluk değerleriyle sonuçlanmıştır.

Bu bölümde bahsedilen tüm metrikler test aşamasında alınmıştır. Farklı veri setlerinde yapılan test sonuçları farklılık gösterebilmektedir.

6. SONUÇ VE ÖNERİLER

Bu bölümde, araştırmadan elde edilen sonuçlar ana hatlarıyla özetlenmiş, projenin genel değerlendirmesi yapılmış ve bu sonuçlara dayalı olarak bilime ve uygulamaya yönelik bir dizi öneri sunulmuştur. Bu sayede, "araştırmanın sonuçları nelerdir ve bu sonuçlara göre neler yapılmalıdır?" sorularına cevap verilmiştir.

Bu projenin temel amacı, HDFS (Hadoop Distributed File System) log verilerindeki anomalileri etkin bir şekilde tespit edebilen, ölçeklenebilir ve güvenilir bir makine öğrenimi modeli geliştirmektir. Elde edilen bulgular, projenin bu hedefe başarıyla ulaştığını göstermektedir. Hibrit (RF+KNN) tekil modellerin bazı hatalarını dengeleyerek kararlılığı artırmıştır; yine de performans tavanı, ardışıklık bilgisinin eksikliğinden etkilenmektedir.

Ancak bazı durumlar göz önüne alınmalıdır. Log_count, seviye dağılımları, benzersiz bileşen/kelime çeşitliliği ile TF-IDF (uni-/bi-gram) birleşimi, etiketli HDFS yapılarında hızlı bir yapı oluşturuldu. Eğitim maliyeti ve yorumlana bilirlik açısından avantajlıdır; ardışıklık bilgisini doğrudan modellemediği için bazı sıralama-kaynaklı anomalileri kaçırabilir. En iyi modelin şu anda RF+KNN yani hibrit model olduğunu söylemek yanlış olmaz. Çünkü hibrit model diğer modellere nazaran yüksek ya da düşük verili setlerde

Projede kullanılan dinamik özellik çıkarma yaklaşımı, doğruluk oranını olumsuz etkileme potansiyeline sahiptir. Bu durum, veri setinin yapısına bağlı olarak model performansının değişkenlik gösterebileceğini ortaya koymaktadır. Bu nedenle, kullanıcıların projeyi yeni veri setleri üzerinde kullanırken her bir modeli ayrı ayrı test ederek en yüksek performansı veren modeli belirlemesi önerilir.

Projede, “ait-aecid/anomaly-detection-log-datasets/hdfs_loghub” [31] veri seti test işlemleri için kullanıldı. Bu veri setinde “hdfs_test_abnormal, hdfs_test_normal, [31] hdfs_train.txt” adında 3 adet veri seti bulunmaktadır. Normal verilerden oluşturulmuş “hdfs_test_normal” veri setinde, 14.178 adet normal log bloğu bulunur. Anomali verilerden oluşturulmuş “hdfs_test_abnormal” veri setinde, 16.838 adet anomali log bloğu bulunur.

Model	hdfs_test_abnormal	hdfs_test_normal
XGBoost	%100	%100
CatBoost	%87	%93
LightGBM	%100	%96
RandomForest	%100	%100
KNN	%85	%100
Hibrit (KNN+RandomForest)	%100	%100

Tablo 3 Doğruluk Oranları.

Tablo 3’te görüldüğü üzere bu proje özellikle Hibrit (KNN ve RandomForest), XGBoost, CatBoost modelleri iyi performans göstermiştir.

Proje / Makale	Kullanılan Metod / Model	HDFS için Rapor Edilen Doğruluk / F1 Skoru / Diğer Performans Ölçüsü
Siamese Network (Hashemi & Mäntylä, 2021) [22]	Siamese Network + LSTM.	HDFS: 0.996 F1

LAnoBERT [32]	Masked Language Modeling (BERT), parser-free.	HDFS: 0.8232 F1
BERT-Log [33]	Önceden eğitilmiş BERT ile log mesajlarının gömme (embedding) temsili.	HDFS: ≈%96,9 F1
Bu Proje	Ağaç Temelli Takviye Öğrenmesi (XGBoost) ve Hibrit temelli yaklaşım.	HDFS: ≈%97 F1

Tablo 4 Aynı veri seti ile yapılan test sonuçları.

Tablo 4, bu projenin HDFS veri seti üzerinde elde ettiği sonuçları, literatürdeki diğer çalışmaların başarılarıyla karşılaştırmaktadır. Bu karşılaştırma, projemizin performansının diğer projelere kasten iyi olduğu görülebilmektedir.

Araştırma bulgularımızda, XGBoost gibi tekil modellerin yüksek doğruluk oranlarına ulaşmasına rağmen, farklı veri setleri üzerinde tutarlı ve stabil sonuçlar vermediği gözlemlenmiştir. Buna karşılık, Hibrit yaklaşımın projenin en temel hedeflerinden olan yüksek güvenilirlik ve tutarlılık beklentisini karşıladığı, birçok farklı veri seti kullanılarak yapılan testlerle kanıtlanmıştır.

Bu sonuçlara dayanarak, benzer anomali tespiti projeleri üzerinde çalışacak araştırmacılara ve geliştiricilere, tekil modeller yerine hibrit modelleme yaklaşımını kullanmaları şiddetle tavsiye edilir. Hibrit modellerin farklı algoritmaların güçlü yönlerini birleştirerek anomali tespitinin doğruluğunu ve genel performansını önemli ölçüde artırdığı bu çalışma ile ortaya konulmuştur.

7. BİRLEŞMİŞ MİLLETLER SÜRDÜRÜLEBİLİR KALKINMA AMAÇLARI İLE İLİŞKİLENDİRME

Birleşmiş Milletler Sürdürülebilir Kalkınma Amaçları Listesi ve Açıklamaları:

1. Yoksulluğa Son (No Poverty)

Her türlü yoksulluğun, her yerde sona erdirilmesi.

2. Açlığa Son (Zero Hunger)

Açlığın sona erdirilmesi, gıda güvenliğinin sağlanması, beslenmenin iyileştirilmesi ve sürdürülebilir tarımın desteklenmesi.

3. Sağlık ve Kaliteli Yaşam (Good Health and Well-being)

Her yaşta sağlıklı yaşamın güvence altına alınması ve refahın desteklenmesi.

4. Nitelikli Eğitim (Quality Education)

Herkes için kapsayıcı ve adil kaliteli eğitimin sağlanması ve yaşam boyu öğrenme fırsatlarının teşvik edilmesi.

5. Toplumsal Cinsiyet Eşitliği (Gender Equality)

Kadınlara ve kız çocuklarına yönelik her türlü ayrımcılığın sona erdirilmesi ve eşitliğin sağlanması.

6. Temiz Su ve Sanitasyon (Clean Water and Sanitation)

Herkes için suya ve sanitasyona erişimin sağlanması ve sürdürülebilir yönetimi.

7. Erişilebilir ve Temiz Enerji (Affordable and Clean Energy)

Herkes için uygun fiyatlı, güvenilir, sürdürülebilir ve modern enerjiye erişimin sağlanması.

8. İnsana Yakışır İş ve Ekonomik Büyüme (Decent Work and Economic Growth)

Sürdürülebilir ekonomik büyüme, tam ve üretken istihdam ve insana yakışır işler.

9. Sanayi, Yenilikçilik ve Altyapı (Industry, Innovation and Infrastructure)

Dirençli altyapıların inşası, kapsayıcı ve sürdürülebilir sanayileşmenin teşvik edilmesi ve yenilikçilik.

10. Eşitsizliklerin Azaltılması (Reduced Inequalities)

Ülkeler içinde ve ülkeler arasında eşitsizliklerin azaltılması.

11. Sürdürülebilir Şehirler ve Topluluklar (Sustainable Cities and Communities)

Şehirlerin ve yerleşim yerlerinin kapsayıcı, güvenli, dirençli ve sürdürülebilir hale getirilmesi.

12. Sorumlu Tüketim ve Üretim (Responsible Consumption and Production)

Sürdürülebilir tüketim ve üretim kalıplarının sağlanması.

13. İklim Eylemi (Climate Action)

İklim değişikliği ve etkileriyle mücadele için acil önlemler alınması.

14. Sudaki Yaşam (Life Below Water)

Okyanusların, denizlerin ve deniz kaynaklarının korunması ve sürdürülebilir kullanımı.

15. Karasal Yaşam (Life on Land)

Karasal ekosistemlerin sürdürülebilir kullanımı, ormanların korunması, çölleşmeyle mücadele ve biyolojik çeşitliliğin korunması.

16. Barış, Adalet ve Güçlü Kurumlar (Peace, Justice and Strong Institutions)

Barışçıl ve kapsayıcı toplumların teşviki, adalet erişimi ve her düzeyde etkin, hesap verebilir kurumların inşası.

17. Amaçlar için Ortaklıklar (Partnerships for the Goals)

Sürdürülebilir kalkınma için küresel ortaklıkların güçlendirilmesi.



Şekil 7.0.1 Birleşmiş milletler sürdürülebilir kalkınma amaçları.

<https://turkiye.un.org/tr/sdgs>

Çalışmanın Birleşmiş Milletler Sürdürülebilir Kalkınma Amaçları Kapsamında Değerlendirilmesi

Bu proje, temel olarak bilgi teknolojileri altyapısının güvenliğini ve sürdürülebilirliğini

sağlamaya odaklandığı için, Birleşmiş Milletler Sürdürülebilir Kalkınma Amaçlarından özellikle Amaç 8 ve Amaç 9 doğrudan ve anlamlı katkılar sağlamaktadır.

7.1 Amaç 8: İnsana Yakışır İş ve Ekonomik Büyüme (Decent Work and Economic Growth):

Bu proje, sürdürülebilir ekonomik büyüme ve üretken istihdamın temelini oluşturan dijital altyapının güvenilirliğini artırarak bu amaca hizmet etmektedir. İlgili alt hedefler 8.1,8.2 ve 8.3 tür. Günümüzde modern devletler veriye dayalı iş modelleri ve kesintisiz dijital hizmetler üzerine kuruludur. HDFS loglarındaki anomali tespiti, büyük veri sistemlerinde oluşabilecek kritik arızaların, veri kayıplarının veya siber saldırıların önüne geçilmesini sağlar. Bu sayede şirketler ve kurumlar için süreklilik ve üretkenlik devam edebilir. Beklenmedik sistem çöküşlerinden kaynaklanan finansal kayıplar ve ekonomik zararlar en aza indirilir. Daha güvenilir bir dijital altyapı, yeni ve insana yakışır iş kollarının gelişimi için zemin hazırlar. Projenin sağladığı fayda, hem yerel işletmelerin ekonomik istikrarını korumaya yardımcı olur hem de ulusal düzeyde dijital ekonominin dirençliliğini güçlendirir.

7.2 Amaç 9: Sanayi, Yenilikçilik ve Altyapı (Industry, Innovation and Infrastructure):

Bu çalışma, modern sanayinin ve dijital altyapının en kritik bileşenlerinden biri olan veri yönetimi sistemlerini daha dayanıklı ve güvenilir hale getirmeyi amaçladığı için Amaç 9'un en önemli hedeflerine doğrudan katkı sunmaktadır. İlgili alt hedefler 9.1, 9.4 ve 9.5'tir. Bu proje, makine öğrenimi ve yapay zeka gibi yenilikçi teknolojileri kullanarak, HDFS gibi kritik altyapı bileşenlerinin zayıf noktalarını tespit ederek sistemin dayanıklılığını artırır [9.1]. Geliştirilen model ve metodoloji, endüstriyel sektörlerin teknolojik kabiliyetlerini yükseltmek için kullanılabilir. Bu proje, üniversite-sanayi iş birliğinin bir örneği olarak, bilimsel araştırma ve geliştirme alanına katkı sağlar [9.5]. Hibrit model yaklaşımı gibi yenilikçi çözümlerin tespiti, benzer sorunlara çözüm arayan gelecek projeler için bir referans noktası oluşturur [9.5].

KAYNAKLAR

- [1] He, P., et al. (2016). A novel log parsing approach for large-scale log analysis. In Proceedings of the 25th International Joint Conference on Artificial Intelligence (IJCAI)
- [2] Breiman, L. (2001). Random Forests. Machine Learning, 45(1), 5-32.
- [3] Ahammad, S. H., Islam, M. T., & Khan, M. J. A. (2021). A comprehensive review on log anomaly detection. Journal of Network and Computer Applications, 178, 102986. <https://doi.org/10.1016/j.inca.2021.102986>
- [4] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (ss. 785-794). ACM.
<https://doi.org/10.1145/2939672.2939785>
- [5] Du, G., et al. (2017). DeepLog: Anomaly detection and diagnosis from system logs through deep learning. In Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (ss. 1109-1124). ACM.
<https://doi.org/10.1145/3133956.3134005>
- [6] Garcia, S., et al. (2012). An empirical comparison of machine learning algorithms for network intrusion detection. In Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP). SCITEPRES
- [7] Goldstein, M., & Dengel, A. (2012). Towards a comprehensive evaluation of unsupervised anomaly detection. In International Conference on Information Systems Security and Privacy (ICISSP). AAAI Press.
- [8] Ke, G., et al. (2017). LightGBM: A highly efficient gradient boosting decision tree. In Advances in Neural Information Processing Systems 30 (NIPS 2017) (ss. 3146-3154).
- [9] Li, J., et al. (2020). LogBERT: Log anomaly 37 detection via BERT. In Proceedings of the 29th ACM Joint European Software Engineering Conference and Symposium

on the Foundations of Software Engineering (ESEC/FSE). ACM.
<https://arxiv.org/abs/2005.04505>

[10] Aplet, G.H., Johnson, N., Olson, J.T. ve Sample, V.A. (1993). Defining Sustainable Forestry, Conclusion: Prospects for a Sustainable Future, Eds.; Aplet GH, Johnson N, Olson, JT ve Sample VA; Island Press, Washington D.C.,309-314.

[11] Prokhorenkova, L., et al. (2018). CatBoost: Unbiased boosting with categorical features. In Advances in Neural Information Processing Systems 31 (NeurIPS 2018).

[12] Shiravi, A., Shiravi, H., & Tavallaei, R. (2012). Toward a new generation of intrusion detection datasets. In Proceedings of the International Conference on Information Systems Security and Privacy (ICISSP). SCITEPRESS.

[13] Sommer, R., & Paxson, V. (2010). Outside the closed world: On live traffic analysis of encrypted network protocols. In 2010 IEEE Symposium on Security and Privacy (SP) (ss. 119-124). IEEE. <https://doi.org/10.1109/SP.2010.16>

[14] Loglizer. (t.y.). GitHub. <https://github.com/logpai/Loglizer>

[15] LogLead. (t.y.). GitHub. <https://github.com/LogLead/LogLead>

[16] WraySmith/log-anomaly. (t.y.). GitHub. <https://github.com/WraySmith/log-anomaly>

[17] Du, M., Li, F., Shi, G., & Liu, J. (2017). DeepLog: Anomaly detection and diagnosis from system logs through deep learning. Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 1285-1298.

[18] He, S., Liu, J., Chen, H., & Zhou, B. (2020). A comprehensive study of machine learning techniques for log-based anomaly detection. IEEE Transactions on Knowledge and Data Engineering, 32(11), 2210-2223.

[19] Guo, S., Ma, S., Liu, Z., Shi, G., & Lin, C. J. (2022). LogBERT: Log anomaly detection via BERT-based masked language model. IEEE Transactions on Knowledge and Data

Engineering, 34(1), 1-13.

[20] Le, V. D., & Zhang, Y. (2022). LogLead: A simple yet effective log-based anomaly detection method. 2022 IEEE International Conference on Systems, Man, and Cybernetics (SMC), pp. 209-216.

[21] Du, M., Li, F., Shi, G., & Liu, J. (2017). DeepLog: Anomaly detection and diagnosis from system logs through deep learning. Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp. 1285-1298.

[22] Hashemi, H., & Mäntylä, M. (2021). Log anomaly detection with a Siamese network. Proceedings of the 2021 IEEE 11th International Conference on Information Technology and Systems (ICITS), pp. 1-6.

[23] LogHub. (t.y.). GitHub. <https://github.com/logpai/loghub> .

[24] Xie, M., Wang, Y., & Zhang, L. (2020). Log Analysis Using Regular Expressions: Importance and Applications. Journal of Software Engineering and Applications, 13(4), 123-135. DOI: 10.4236/jsea.2020.134012 .

[25] **Chandola, V., Banerjee, A., & Kumar, V.** (2009). Anomaly Detection: A Survey. *ACM Computing Surveys*, 41(3), 1-58.

[26] **Goldstein, M., & Dengel, A.** (2012). Histogram-based Outlier Score (HBOS): A fast Unsupervised Anomaly Detection Algorithm. *Proceedings of the KI 2012: Poster and Demo Track*, 59-63.

[27] **Logpai. (t.y.).** *logpai/deep-loglizer: A deep learning toolkit for log-based anomaly detection.* GitHub. <https://github.com/logpai/deep-loglizer>

[28] **Thijsvanede. (t.y.).** *Thijsvanede/DeepLog: PyTorch implementation of Deeplog: Anomaly detection and diagnosis from system logs through deep learning.* GitHub. <https://github.com/Thijsvanede/DeepLog>

- [29] **Umer, M., Tariq, A., Rehman, Z., & Khan, M. A.** (2020). LogRobust: A hybrid log-based anomaly detection method. *IEEE Access*, 8, 105650-105663.
- [30] **LogHub. (t.y.).** *Hadoop Distributed File System Log Data*. Kaggle.
[LogHub - Hadoop Distributed File System Log Data](#)
- [31] **AIT-AECID. (t.y.).** *ait-aecid/anomaly-detection-log-datasets: Analysis scripts for log data sets used in anomaly detection*. GitHub. <https://github.com/ait-aecid/anomaly-detection-log-datasets>
- [32] **Lee, Y., Kim, J., & Kang, P.** (2021). LAnoBERT: System Log Anomaly Detection based on BERT Masked Language Model. *arXiv preprint arXiv:2111.09564*.
- [33] **Chen, S., & Liao, H.** (2022). BERT-Log: Anomaly Detection for System Logs Based on Pre-trained Language Model. *Applied Artificial Intelligence*, 36(1).